AP 11 : SSH, VPN et Fail2ban

Bchi Aliya-GOODMAN Mathéo

lien gant:
https://docs.google.com/spreadsheets/d/12SJmeMe_uRSRjKyYKNjC6nryp1X3vQVVMd1LG3rIP4g/edit?usp=sharing

lien schéma réseau:
https://drive.google.com/file/d/1OUvV3dj2cvojEKEhfht_rPbyXuJPmcgI/view?usp=sharing

Pré-requis :

J'ai procédé à la création d'une nouvelle machine virtuelle destinée à héberger le serveur VPN. Cette VM servira de plateforme pour l'installation, la configuration et la mise en place des services VPN.

```
root@Srv-vpn:~#
```

J'ai affiché l'adresse IP de la machine à l'aide de la commande `ip a` afin d'identifier son interface réseau et vérifier sa configuration.

```
root@Srv-vpn:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:0b:62:4a brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24110b624a
    inet 192.168.13.18/24 brd 192.168.13.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe0b:624a/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Le serveur VPN a été configuré sur le **VLAN 450**, afin de l'isoler et de le séparer logiquement du reste du réseau.

2557 (Srv-vpn)	Network Device (net0)	virtio=BC:24:11:0B:62:4A,bridge=vmbr0,tag=450

J'ai ajouté une entrée DNS pour le serveur VPN (**srv-vpn**) afin qu'il soit correctement référencé et accessible par son nom dans le réseau.

```
Srv-vpn IN      A       192.168.13.18
```

J'ai effectué une vérification à l'aide de la commande `nslookup` afin de confirmer que le serveur **srv-vpn** est correctement résolu par le DNS. La commande retourne bien l'adresse IP associée, ce qui valide le bon référencement DNS du serveur.

```
root@clonedns:~# nslookup Srv-vpn
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   Srv-vpn.menuimetal.fr
Address: 192.168.13.18
```

J'ai procédé à la remontée du serveur VPN dans GLPI afin d'assurer son inventaire et son suivi. Le serveur apparaît désormais dans la liste des équipements recensés.

Srv-vpn	Entité racine	QEMU	QEMU Standard PC (I440FX + PIIX, 1996)	Debian GNU/Linux 13 (trixie)	2025-11-28 11:39	QEMU Virtual CPU version 2.5+

J'ai intégré le serveur VPN dans Nagios afin d'assurer sa supervision. Le serveur est maintenant surveillé, permettant de suivre son état, ses services et ses performances en temps réel.

srv-vpn	Ping	OK	11-28-2025 13:55:47	0d 0h 5m 0s+	1/3	PING OK - Packet loss = 0%, RTA = 0.58 ms
	Port IPsec 4500	UNKNOWN	11-28-2025 14:01:02	0d 0h 0m 3s	3/3	With UDP checks, a send/expect string must be specified.
	Port IPsec 500	UNKNOWN	11-28-2025 14:00:16	0d 0h 2m 49s	2/3	With UDP checks, a send/expect string must be specified.
	Port OpenVPN	CRITICAL	11-28-2025 13:59:31	0d 0h 1m 34s	1/3	connect to address 192.168.13.18 and port 1194: Connexion refusée
	SSH	OK	11-28-2025 14:00:46	0d 0h 5m 0s+	1/3	SSH OK - OpenSSH_10.0p2 (protocol 2.0)

MISSION 1: Le protocole SSH

1] Installation et paramétrage du serveur

J'ai choisi d'utiliser un serveur Linux basique nommé *srv-ssh*, que j'ai cloné afin d'éviter d'utiliser les machines Nagios et GLPI. Ce serveur dispose de l'adresse IP 192.168.13.7.

```
root@Srv-ssh:~#
```

Installation du serveur SSH réalisée avec la commande : `apt install openssh-server -y`, ce qui permet d'activer l'accès SSH sur la machine.

```
root@Srv-ssh:~# apt install openssh-server -y
openssh-server est déjà la version la plus récente (1:10.0p1-7).
openssh-server passé en « installé manuellement ».
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libcbor0.8                libpython3.11-minimal     python3-pkg-resources
  libcurl3t64-gnutls        libpython3.11-stdlib      python3-pycurl
  libfuse2                  perl-modules-5.36         python3-pyparsing
  libicu72                  python3-autocommand       python3-pysimplesoap
  libldap-2.5-0             python3-httplib2          python3-six
  libngtcp2-16              python3-inflect           python3-typeguard
  libngtcp2-crypto-gnutls8  python3-jaraco.context    python3-typing-extensions
  libnsl2                   python3-jaraco.functools  python3.11
  libperl5.36               python3-more-itertools    python3.11-minimal
Veuillez utiliser « apt autoremove » pour les supprimer.

Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 25
```

Le fichier de configuration principal est sshd_config :

```
"/etc/ssh/sshd_config" 124L, 3423B                                    29,0-1
```

Modification du fichier de configuration SSH en ajoutant le port 2222 et en désactivant l'accès root avec l'option `PermitRootLogin no`.

```
Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

Question : Selon ces documentations, quelle est la différence entre les options prohibit-password et forced-commands-only ?

Les options **prohibit-password** et **forced-commands-only** ont deux objectifs différents dans la configuration SSH. L'option *prohibit-password* interdit toute connexion utilisant un mot de passe, ce qui force l'utilisation d'une clé SSH pour s'authentifier, tout en permettant à l'utilisateur d'accéder normalement à un shell.
À l'inverse, l'option *forced-commands-only* limite strictement l'utilisateur à l'exécution d'une commande prédéfinie dans le fichier *authorized_keys*. Même avec une clé SSH valide, l'utilisateur ne peut pas ouvrir une session interactive : seule la commande imposée est exécutée. Ainsi, *prohibit-password* renforce la sécurité de la connexion, tandis que *forced-commands-only* encadre et restreint les actions autorisées après la connexion.

Le service SSH a ensuite été redémarré afin de prendre en compte les modifications apportées à la configuration.

```
root@Srv-ssh:~# systemctl restart ssh
```

Après le redémarrage, j'ai vérifié le statut du service SSH et il est bien fonctionnel.

```
root@Srv-ssh:~# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-11-25 14:57:27 CET; 1min 1s ago
 Invocation: 1dc80a1a53d24286bae74bd0835023ab
```

À l'aide de la commande `netstat -tulpn | grep -i ssh`, j'ai vérifié que le service SSH est bien en écoute (LISTEN). Les résultats confirment que le service attend des connexions à la fois en TCP pour l'IPv4 et en TCP6 pour l'IPv6, comme demandé

```
root@Srv-ssh:~# netstat -tulpn | grep -i ssh
tcp        0      0 0.0.0.0:2222            0.0.0.0:*               LISTEN      860/sshd: /usr/sbin
tcp6       0      0 :::2222                 :::*                    LISTEN      860/sshd: /usr/sbin
```

2] Se connecter à un ordinateur distant via SSH avec un mot de passe

Je me rends ensuite sur ma machine cliente *cltlinux2*, qui dispose de l'adresse IP 192.168.13.45.

```
sio@cltlinux2:~$
```

J'installe ensuite le paquet *openssh-client* sur la machine cliente afin de pouvoir établir une connexion SSH vers le serveur.

```
root@cltlinux2:~# apt install openssh-client -y
openssh-client est déjà la version la plus récente (1:10.0p1-7).
Les paquets suivants ont été installés automatiquement et ne sont p
```

J'ai ensuite établi une connexion SSH sur le serveur cltlinux2 pour poursuivre les opérations nécessaires.

```
root@cltlinux2:~# ssh srv-ssh@192.168.13.7 -p 2222
srv-ssh@192.168.13.7's password:
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
srv-ssh@srv-ssh:~$
```

Lors de la première connexion SSH, la clé publique du serveur n'étant pas encore connue, SSH a demandé de confirmer l'identité du serveur afin de prévenir toute attaque de type 'man-in-the-middle'. En validant par 'yes', la clé du serveur a été ajoutée au fichier ~/.ssh/known_hosts de la machine cliente, ce qui évitera ce message lors des prochaines connexions.

J'ai affiché l'adresse IP de la machine virtuelle afin de vérifier la connexion effective au serveur SSH

```
srv-ssh@srv-ssh:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
qlen 1000
    link/ether bc:24:11:2e:6d:50 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24112e6d50
    inet 192.168.13.7/24 brd 192.168.13.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe2e:6d50/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
srv-ssh@srv-ssh:~$
```

Je me suis ensuite déconnectée du serveur cltlinux2.

```
srv-ssh@srv-ssh:~$ exit
déconnexion
Connection to 192.168.13.7 closed.
root@cltlinux2:~#
```

Nous avons ensuite tenté de nous connecter avec l'utilisateur root, mais l'accès nous a été refusé. La connexion SSH directe avec root est désactivée sur le serveur via `PermitRootLogin no` afin de prévenir les risques de sécurité, notamment les attaques par force brute. Il est donc nécessaire de se connecter avec un utilisateur standard et d'utiliser `sudo` pour obtenir les privilèges administratifs. Le message d'erreur reçu lors de la tentative de connexion root est normal et confirme l'application de cette règle de sécurité.

```
root@cltlinux2:~# ssh root@192.168.13.7 -p 2222
root@192.168.13.7's password:
Permission denied, please try again.
root@192.168.13.7's password:
Permission denied, please try again.
root@192.168.13.7's password:
```

Par ailleurs, j'ai ajouté `PasswordAuthentication yes` dans le fichier `/etc/ssh/sshd_config`. Cette modification permet aux utilisateurs normaux de se connecter via mot de passe, mais ne change rien pour le compte root qui reste bloqué à cause de `PermitRootLogin no`. Cela correspond exactement à ce qui était attendu : sécuriser le compte root tout en permettant l'accès SSH aux utilisateurs standards.

Ensuite, la connexion SSH depuis la VM cliente fonctionne correctement avec l'utilisateur `srv-ssh`, et la prochaine étape consistera à configurer l'authentification par clé publique et la redirection des logs vers le serveur Rsyslog central.

```
# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication yes
```

3] Se connecter à un ordinateur distant via SSH avec une clef publique

Authentification par clé publique SSH

L'authentification par clé publique permet de se connecter à un serveur SSH sans mot de passe, en utilisant une paire de clés :

- Clé privée : reste sur la machine cliente et doit être protégée par une passphrase.
- Clé publique : est copiée sur le serveur et permet de vérifier que la clé privée correspond.

Cette méthode utilise le cryptage asymétrique, ce qui renforce la sécurité : même si quelqu'un connaît votre mot de passe système, il ne pourra pas se connecter sans la clé privée.

J'ai créé un compte utilisateur portant mon nom de famille (sans espace ni accent) avec un mot de passe identique (sans espace ni accent).

```
root@cltlinux2:~# adduser bchi
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour bchi
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
        NOM []:
        Numéro de chambre []:
        Téléphone professionnel []:
        Téléphone personnel []:
        Autre []:
Is the information correct? [Y/n]
root@cltlinux2:~#
```

```
root@srv-ssh:~# adduser bchi
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour bchi
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
        NOM []:
        Numéro de chambre []:
        Téléphone professionnel []:
        Téléphone personnel []:
        Autre []:
Is the information correct? [Y/n]
```

Je me suis connectée au serveur srv-ssh depuis cltlinux2 via SSH.

```
root@srv-ssh:~# ssh bchi@192.168.13.45
The authenticity of host '192.168.13.45 (192.168.13.45)' can't be established.
ED25519 key fingerprint is SHA256:7tpoM2981cyrS78FAQFyPhybIpTi9YgY/RFInWEzSms.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.13.45' (ED25519) to the list of known hosts.
bchi@192.168.13.45's password:
Linux cltlinux2 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
bchi@cltlinux2:~$
```

J'ai généré les clés SSH (publiques et privées) pour la version SSH2 en utilisant la commande `ssh-keygen -t rsa` et en définissant la passphrase Sio25!.

```
root@cltlinux2:~# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:+hM5iaFSd/4Qoahom2Bw9k9YKyM6Md3e9PHM6U6YWzA root@cltlinux2
The key's randomart image is:
+---[RSA 3072]----+
|                 |
|        .        |
|     . . .       |
|. o o = o        |
|.= = = *E+       |
|=o= B =.O=       |
|++o+ B..oXo.     |
|oo  . o.o+*      |
| .      o+o      |
+----[SHA256]-----+
root@cltlinux2:~#
```

Sur la machine virtuelle cliente, j'ai vérifié, depuis le répertoire personnel de l'utilisateur (mon nom de famille), la bonne création des fichiers dans le dossier `.ssh` à l'aide de la commande `ls -l .ssh`. Le fichier portant l'extension `.pub` correspond à la clé publique, tandis que l'autre correspond à la clé privée :

- `id_rsa` → clé privée
- `id_rsa.pub` → clé publique

```
root@cltlinux2:~# ls -l .ssh
total 16
-rw------- 1 root root 2655 25 nov.  16:34 id_rsa
-rw-r--r-- 1 root root  568 25 nov.  16:34 id_rsa.pub
-rw------- 1 root root  978 25 nov.  15:58 known_hosts
-rw-r--r-- 1 root root  142 25 nov.  15:08 known_hosts.old
root@cltlinux2:~#
```

J'ai affiché le contenu de chaque clé à l'aide de la commande `cat` afin d'en observer le contenu, bien que celui-ci soit incompréhensible.

Dans un premier temps, j'ai testé la clé privée `id_rsa`.

```
root@cltlinux2:~# cat ~/.ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABDZ3rRT6l
aa4RbFfBdun6pfAAAAGAAAAAEAAAGXAAAAB3NzaC1yc2EAAAADAQABAAABgQDbeeynLlb/
ES1lqIyzYDoSqh50w4DZ0ubRcjDQpSrH+fJo5TUbzUusMLEv6TXSY0XHxiY+/IqZ6CrEmD
i8m1z6kCbNmcDc9NXfmWK/WHBbgEIr7khoCoh2ZmerEWdXY+5QC8W3cgxskCCyX3MSYUyo
/esaI3AZXvf5dX5JArXTaJS4VLsfRYWMXE50B6wFBC6ua1LnzjIyiIOjsX7HDoHSwgRS/e
WVMUI1qDJB1MwQm/fC9b7Ju+RaESyqtlzrWbXLowMWCYkFAGjWBhA3pJCRP/sHThdk8vCP
Ry84aUrYsAdx3NxQiRbmbfMXkHMiY9H3HVaH+tyHb8RQ+DJwDxWU5b5gCLMozFOqN/aweH
76GZLXWs/W3xneu4JnJvtAzAs6+QHTs94KMYFW4HryDLX07yk5cVfOaXG5K7FfnjfDtsKz
9vh6EN+DbXx87SbT0Ea4vTx0gIfLOvImD1PsTFo0X1dK1Cb1JJJ3ZJBHQofD6n8AYvIMey
7yq1YZ8dvZMcsAAAWQ9gO1C7VtEGbyG/FKNiS1Bjj4CbqcIjVsC/y+joDX69dM/UqT8L01
r0GIHZiOKja5gKjkrYE479T/mdyGQseIBhdZ4edfoemyWdBzu+JfC/YMhAc+O7nmYtI/W6
jiZa/p4NLFUhVfD6Xi1DBfY+CnDJU33wSfJhDuaqUf1c2IArjWk6BDcfVV0IYn8FN2ZBLc
X/RekfzTPOhcssr/F68roBGf9tTt1bySArwckEYlztAfICSqafGHVdFXpw7ZOBlleAP7o
dKkGxU492OyNEkoggVATYKIB8AHmppMs1ZJgdB+YVeJ45JUVSCnUkSfY6osFsyQ1v/NURP
GIlow7ZR34CYPpXYizeRnQo5dePy6CCMngIghMhJ7RNZ6ywVSuFJexvrsnBxqeYZ4VXHS9
r5qzjhbJdiy1M+dIVURjy4gf5maEjEQm5/FPjC7zI0nsRvw7t+kyuxf2miDpLPeifEsE4n
PPPl+2s4F6uNqVwa+L8SKHLSdCVjDgtRkq9qpd46SDoloRFDr7poOywuHhIn+WEFB9BE8E
JHdpCqF6nL05Ia4s+tSsgXegOL86bcUfxYX7bT9QDtqwBdTnpQLe8Cg/3GDFTsvi8D7/+p
0p8tJ4/txY01gD5Vf4nUerpMNP43jALvRFbACydhmJg2PkvX74qFDcV0qOnrdK4tGMkvfP
amptcazmgbERdRM0vUgvnyGrACg4PnibghqnFWMdwEfuahPQb89A265WXISepnKkWx769F
WnJJi9AnDSl+/HxbKLQq6TKomh6U2bb/oDQ5eayjTR+sUwWvpxP0CFxbKri+r7BtK4t10z
hZfiyeja/jEuRerDSGUkJY3ARxKm9b8o7w07f/klQdubbX0fawzKAopE9P3s3iFh79J3hq
CcEzIWOiFWiFbTZgruqpPTK0U9zCkUPGrvfKrBm103vkFwW7d/uYVAo8EORwltIkYMuwFG
4KaSlxFepfpqlwPXuVketaLF66Bi06IZaA4N5dpNtLv4iSE2JRzQvIjOVyWenL5oshlgm9
hdGAXZxIwcpH+HE4XMT8TVuF8kS72chwgkMzEMks6hSlv7rGz0Dqxa9r+NL7FwxFDZr4lw
tJlln66vPZsEg5mEzOJpEyDDel5Uhz8AiecLJbSDOkVbf8xPHb4wlaz0iRxC/TNzEfKGlJ
KKFpX+9TtagwF05OhdwVvIcMN8jfh/tgQclUeArDztRA13LhaQrJ6XfWPLuZYATVmc5jlZ
LGJeo+kzAeq9C21RUH0uNT4FR2RhqxW1rdvYbV5mAW31IpfCl6C/xJzTzuR46Zu1P8vosv
IquF3yT2n7g8X/vyl/FhM2vuedFb8svNODkcZQcp0j5TeV6WIekOEbP+YwOIPwODfVykAp
E3er8KdJLfMDzv9UpXUkwgd5CsX6arGuqY3+z7yHXUSgwqpL/IFzOJ67k4ELKoV5RcLyRU
7e5tET+W4zphK56ZNcMJsqRN9YkhHk513e7eW42Mt0i9sulym3jPv8blp9140qdu52w5wZ
S2gz8H4L4Y7Yw5ZE4noMY3jhFZMookWQO0AzDBeJA/jlqhOFCujumwc8C9P3S8rrEkdCVs
+MfcDI88Hhjm+rKIvQc57Jb4DZj77ObnxCAU8tlXcNBiPphjwku60dwRmwcE6I39q4S7+J
A98hPSs8cdHzp01JE0StOCMeYVZpwxKcEFJBDLEE0NXPhHPtIPOXs/udpxw+/PhFutrphP
yeUxp3KKuqLzqPJofKz8c4rnuawV4IrAO1TX5fwcYHX/S6oukXk4OOelml524S7pWO70V7
deftyjqMwjr35MP6y+wiDzyql39+NpupN2uI8WKACX7xsh/XjWevrLeIgZahyX6hsj4FfM
sgapiNdhAOfu8Mv2Tit4H32T7Bo=
-----END OPENSSH PRIVATE KEY-----
root@cltlinux2:~#
```

J'ai ensuite testé la clé publique `id_rsa.pub`.

```
root@cltlinux2:~# cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDbeeynLlb/ES1lqIyzYDoSqh50w4DZ0ubRcjDQpSrH+fJo5TUbzUu
sMLEv6TXSY0XHxiY+/IqZ6CrEmDi8m1z6kCbNmcDc9NXfmWK/WHBbgEIr7khoCoh2ZmerEWdXY+5QC8W3cgxskCCyX3
MSYUyo/esaI3AZXvf5dX5JArXTaJS4VLsfRYWMXE50B6wFBC6ua1LnzjIyiIOjsX7HDoHSwgRS/eWVMUI1qDJB1MwQm
/fC9b7Ju+RaESyqtlzrWbXLowMWCYkFAGjWBhA3pJCRP/sHThdk8vCPRy84aUrYsAdx3NxQiRbmbfMXkHMiY9H3HVaH
+tyHb8RQ+DJwDxWU5b5gCLMozFOqN/aweH76GZLXWs/W3xneu4JnJvtAzAs6+QHTs94KMYFW4HryDLX07yk5cVfOaXG
5K7FfnjfDtsKz9vh6EN+DbXx87SbT0Ea4vTx0gIfLOvImD1PsTFo0X1dK1Cb1JJJ3ZJBHQofD6n8AYvIMey7yq1YZ8d
vZMcs= root@cltlinux2
root@cltlinux2:~#
```

En utilisant la commande *ssh-copy-id* depuis la machine cliente, j'ai copié automatiquement ma clé publique sur le serveur (192.168.13.7) pour l'utilisateur bchi. Le message *Number of key(s) added: 1* indique qu'une clé a été ajoutée dans le fichier *authorized_keys*, ce qui permet désormais une connexion SSH sécurisée par clé publique.

```
root@cltlinux2:~# ssh-copy-id -i ~/.ssh/id_rsa.pub -p 2222 bchi@192.168.13.7
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any tha
t are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it i
s to install the new keys
bchi@192.168.13.7's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /root/.ssh/id_rsa -p 2222 'bchi@192.168.13.
7'"
and check to make sure that only the key(s) you wanted were added.

root@cltlinux2:~#
```

Après avoir généré ma paire de clés SSH sur la machine cliente, j'ai utilisé la commande `ssh-copy-id` pour transférer automatiquement ma clé publique sur le serveur. Cette opération a créé ou complété le fichier `authorized_keys` côté serveur. J'ai ensuite pu me connecter en SSH sur le port 2222 en utilisant ma clé privée, qui n'a demandé que la passphrase Sio25!, confirmant ainsi le bon fonctionnement de l'authentification par clé publique.

```
root@cltlinux2:~# ssh -p 2222 bchi@192.168.13.7
Enter passphrase for key '/root/.ssh/id_rsa':
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_
64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
bchi@srv-ssh:~$
```

J'ai vérifié sur le serveur que ma clé publique était bien copiée dans le fichier `authorized_keys` de l'utilisateur `bchi` et non dans celui de root. Cela permet au serveur d'identifier l'utilisateur qui se connecte via sa clé privée correspondante, garantissant ainsi une authentification sécurisée par clé publique. Cette étape est essentielle dans le TP pour s'assurer que la connexion SSH pourra se faire sans mot de passe, tout en maintenant la sécurité du serveur.

```
root@srv-ssh:~# ls -l /home/bchi/.ssh
cat /home/bchi/.ssh/authorized_keys
total 4
-rw------- 1 bchi bchi 568 25 nov.  16:47 authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDbeeynLlb/ES1lqIyzYDoSqh50w4DZ0ubRcjDQpSrH+fJo5TUbzUu
sMLEv6TXSY0XHxiY+/IqZ6CrEmDi8m1z6kCbNmcDc9NXfmWK/WHBbgEIr7khoCoh2ZmerEWdXY+5QC8W3cgxskCCyX3
MSYUyo/esaI3AZXvf5dX5JArXTaJS4VLsfRYWMXE50B6wFBC6ua1LnzjIyiIOjsX7HDoHSwgRS/eWVMUI1qDJB1MwQm
/fC9b7Ju+RaESyqtlzrWbXLowMWCYkFAGjWBhA3pJCRP/sHThdk8vCPRy84aUrYsAdx3NxQiRbmbfMXkHMiY9H3HVaH
+tyHb8RQ+DJwDxWU5b5gCLMozFOqN/aweH76GZLXWs/W3xneu4JnJvtAzAs6+QHTs94KMYFW4HryDLX07yk5cVfOaXG
5K7FfnjfDtsKz9vh6EN+DbXx87SbT0Ea4vTx0gIfLOvImD1PsTFo0X1dK1Cb1JJJ3ZJBHQofD6n8AYvIMey7yq1YZ8d
vZMcs= root@cltlinux2
root@srv-ssh:~#
```

Je me suis connecté en SSH sur le serveur Linux avec l'utilisateur **sio**.

```
root@cltlinux2:~# ssh -p 2222 sio@192.168.13.7
sio@192.168.13.7's password:
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_
64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Nov 25 14:49:36 2025 from 192.168.13.250
sio@srv-ssh:~$
```

J'ai vérifié l'existence du dossier `.ssh` de l'utilisateur **sio**, qui est bien présent.

```
sio@srv-ssh:~$ cd /home/sio
sio@srv-ssh:~$ ls -a
.  ..  .bash_history  .bash_logout  .bashrc  .profile  .ssh
sio@srv-ssh:~$
```

J'ai utilisé la commande `scp` depuis la machine cliente afin de transférer ma clé publique vers le serveur. La commande a été exécutée de la manière suivante : le fichier `~/.ssh/id_rsa.pub` de la cliente a été copié vers le serveur **sio@192.168.13.7** sous le nom temporaire `tmp_id_rsa.pub`. Il est important de ne jamais lancer `scp` depuis le serveur pour copier sa propre clé vers lui-même.

```
root@cltlinux2:~# scp -P 2222 ~/.ssh/id_rsa.pub sio@192.168.13.7:~/.ssh/tmp_id_rsa.pub
sio@192.168.13.7's password:
id_rsa.pub                                    100%  568   709.5KB/s   00:00
root@cltlinux2:~#
```

Pour vérifier que le transfert s'est correctement effectué, j'ai listé les fichiers sur le serveur à l'aide de la commande `ls` et j'ai constaté la présence du fichier `tmp_id_rsa.pub`.

```
sio@srv-ssh:~$ ls -l ~/.ssh
total 4
-rw-r--r-- 1 sio sio 568 25 nov.  17:03 tmp_id_rsa.pub
sio@srv-ssh:~$
```

J'ai affiché le contenu du fichier `tmp_id_rsa.pub` sur le serveur afin de vérifier qu'il s'agissait bien de la clé publique. Le contenu correspond exactement à celui du fichier `id_rsa.pub` sur la machine cliente.

```
sio@srv-ssh:~$ cat ~/.ssh/tmp_id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDbeeynLlb/ES1lqIyzYDoSqh50w4DZ0ubRcjDQpSrH+fJo5TUbzUu
sMLEv6TXSY0XHxiY+/IqZ6CrEmDi8m1z6kCbNmcDc9NXfmWK/WHBbgEIr7khoCoh2ZmerEWdXY+5QC8W3cgxskCCyX3
MSYUyo/esaI3AZXvf5dX5JArXTaJS4VLsfRYWMXE50B6wFBC6ua1LnzjIyiIOjsX7HDoHSwgRS/eWVMUI1qDJB1MwQm
/fC9b7Ju+RaESyqtlzrWbXLowMWCYkFAGjWBhA3pJCRP/sHThdk8vCPRy84aUrYsAdx3NxQiRbmbfMXkHMiY9H3HVaH
+tyHb8RQ+DJwDxWU5b5gCLMozFOqN/aweH76GZLXWs/W3xneu4JnJvtAzAs6+QHTs94KMYFW4HryDLX07yk5cVfOaXG
5K7FfnjfDtsKz9vh6EN+DbXx87SbT0Ea4vTx0gIfLOvImD1PsTFo0X1dK1Cb1JJJ3ZJBHQofD6n8AYvIMey7yq1YZ8d
vZMcs= root@cltlinux2
sio@srv-ssh:~$
```

J'ai également vérifié simultanément le fichier sur la machine cliente et constaté que le contenu était identique, ce qui confirme que le transfert de la clé publique a été effectué correctement.

```
root@cltlinux2:~# cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDbeeynLlb/ES1lqIyzYDoSqh50w4DZ0ubRcjDQpSrH+fJo5TUbzUu
sMLEv6TXSY0XHxiY+/IqZ6CrEmDi8m1z6kCbNmcDc9NXfmWK/WHBbgEIr7khoCoh2ZmerEWdXY+5QC8W3cgxskCCyX3
MSYUyo/esaI3AZXvf5dX5JArXTaJS4VLsfRYWMXE50B6wFBC6ua1LnzjIyiIOjsX7HDoHSwgRS/eWVMUI1qDJB1MwQm
/fC9b7Ju+RaESyqtlzrWbXLowMWCYkFAGjWBhA3pJCRP/sHThdk8vCPRy84aUrYsAdx3NxQiRbmbfMXkHMiY9H3HVaH
+tyHb8RQ+DJwDxWU5b5gCLMozFOqN/aweH76GZLXWs/W3xneu4JnJvtAzAs6+QHTs94KMYFW4HryDLX07yk5cVfOaXG
5K7FfnjfDtsKz9vh6EN+DbXx87SbT0Ea4vTx0gIfLOvImD1PsTFo0X1dK1Cb1JJJ3ZJBHQofD6n8AYvIMey7yq1YZ8d
vZMcs= root@cltlinux2
root@cltlinux2:~#
```

Sur la machine virtuelle Linux serveur, en mode administrateur, j'ai modifié les directives suivantes dans le fichier `/etc/ssh/sshd_config` :

- `PubkeyAuthentication yes` → active l'authentification par clé publique
- `PasswordAuthentication no` → désactive la connexion par mot de passe
- `AuthorizedKeysFile …` → indique les fichiers contenant les clés publiques autorisées

```
PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
AuthorizedKeysFile      .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication no
#PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication.  Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
-- INSERTION --                                                    42,1          22%
```

J'ai relancé le service SSH avec la commande `systemctl restart ssh` puis vérifié son état avec `systemctl status ssh`. Aucune erreur n'a été détectée, ce qui confirme que les directives ajoutées dans le fichier `sshd_config` ont été correctement prises en compte.

```
root@srv-ssh:~# systemctl restart ssh
root@srv-ssh:~# systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-11-25 17:15:05 CET; 6s ago
 Invocation: 73958a74d6dc460184c30e6c3aaecb3a
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1156 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1159 (sshd)
      Tasks: 1 (limit: 2303)
     Memory: 1.3M (peak: 1.8M)
        CPU: 49ms
     CGroup: /system.slice/ssh.service
             └─1159 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

nov. 25 17:15:05 srv-ssh systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
nov. 25 17:15:05 srv-ssh sshd[1159]: Server listening on 0.0.0.0 port 2222.
nov. 25 17:15:05 srv-ssh sshd[1159]: Server listening on :: port 2222.
nov. 25 17:15:05 srv-ssh systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@srv-ssh:~#
```

Je me suis connecté au serveur distant avec l'utilisateur sio en utilisant la commande `ssh`. La clé publique de mon compte utilisateur (mon nom de famille) ayant été placée sur ce compte, il m'a uniquement été demandé de saisir la passphrase associée à ma clé privée. Ainsi, il n'est plus nécessaire de connaître le mot de passe de l'utilisateur distant pour établir la connexion.

```
root@cltlinux2:~# ssh -p 2222 sio@192.168.13.7
Enter passphrase for key '/root/.ssh/id_rsa':
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_
64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Nov 27 16:25:07 2025 from 192.168.13.45
sio@srv-ssh:~$
```

J'ai vérifié que j'étais bien connecté sur le serveur distant en utilisant la commande `hostname`. Cela permet de confirmer que la session SSH est bien établie sur le serveur et non sur la machine locale.

```
sio@srv-ssh:~$ hostname -I
192.168.13.7
sio@srv-ssh:~$
```

Je me suis ensuite déconnectée du serveur.

```
sio@srv-ssh:~$ exit
déconnexion
Connection to 192.168.13.7 closed.
root@cltlinux2:~#
```

3] La gestion des clefs privées et des passphrases

J'ai lancé un agent SSH, ce qui permet de démarrer un nouveau shell avec l'agent actif.

```
root@cltlinux2:~# ssh-agent /bin/bash
root@cltlinux2:~#
```

Tant que ce shell reste ouvert, il n'est plus nécessaire de retaper la passphrase pour la clé lors des connexions au serveur distant. L'agent SSH conserve la clé en mémoire pour la durée de cette session.

```
root@cltlinux2:~# ssh-add
Enter passphrase for /root/.ssh/id_rsa:
Identity added: /root/.ssh/id_rsa (root@cltlinux2)
root@cltlinux2:~#
```

J'ai testé la connexion au serveur distant sans mot de passe ni passphrase. L'agent SSH mémorisant la clé, la connexion s'est établie directement, sans demander de saisie supplémentaire.

```
root@cltlinux2:~# ssh -p 2222 sio@192.168.13.7
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_
64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Nov 27 16:29:27 2025 from 192.168.13.45
sio@srv-ssh:~$
```

4] Les utilitaires ssh

Transfert de fichiers avec SCP

J'ai effectué un transfert de fichier de la machine virtuelle cliente vers le serveur à l'aide de la commande `scp`. Le fichier local `/root/mon_fichier.txt` a été copié vers l'emplacement distant `/home/sio/mon_fichier.txt` sur le serveur `sio@192.168.13.7`.

```
root@cltlinux2:~# scp -P 2222 /root/mon_fichier.txt sio@192.168.13.7:/home/sio/mon_fichier.
txt
mon_fichier.txt                                       100%   17    18.6KB/s   00:00
root@cltlinux2:~#
```

Pour m'assurer que le transfert s'est bien effectué, j'ai vérifié la présence du fichier sur le serveur. Le fichier a bien été transféré, ce qui confirme la réussite de l'opération.

```
root@cltlinux2:~# ssh -p 2222 sio@192.168.13.7
Linux srv-ssh 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86_
64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Nov 27 16:37:37 2025 from 192.168.13.45
sio@srv-ssh:~$ ls /home/sio/
mon_fichier.txt
sio@srv-ssh:~$
```

J'ai récupéré le fichier depuis le serveur distant vers la machine virtuelle cliente. Cette fois, le fichier `mon_fichier.txt` a été transféré avec succès et enregistré sous le nom `mon_fichier_recup.txt` sur la VM cliente.

```
root@cltlinux2:~# scp -P 2222 sio@192.168.13.7:/home/sio/mon_fichier.txt /root/mon_fichier_
recup.txt
mon_fichier.txt                                       100%   17    10.9KB/s   00:00
root@cltlinux2:~#
```

J'ai vérifié la présence des fichiers sur la machine cliente : le fichier original `mon_fichier.txt` envoyé au serveur et la copie récupérée `mon_fichier_recup.txt`. Les deux fichiers sont bien présents, confirmant que la récupération depuis le serveur a été réussie.

```
root@cltlinux2:~# ls -l /root/
total 40
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Bureau
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Documents
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Images
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Modèles
-rw-rw-r-- 1 root root   17 27 nov.  16:49 mon_fichier_recup.txt
-rw-rw-r-- 1 root root   17 27 nov.  16:43 mon_fichier.txt
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Musique
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Public
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Téléchargements
drwxr-xr-x 2 root root 4096  5 sept. 14:06 Vidéos
root@cltlinux2:~#
```

Transfert de fichiers avec SFTP

SFTP est l'équivalent sécurisé de FTP, fonctionnant via SSH, ce qui permet de chiffrer toutes les communications.

Je me suis connecté depuis ma machine locale vers le serveur distant à l'aide de SFTP. La machine locale envoie et récupère les fichiers. L'opération s'est déroulée avec succès.

```
root@cltlinux2:~# sftp -P 2222 sio@192.168.13.7
Connected to 192.168.13.7.
sftp>
```

J'ai effectué un test de commandes SFTP pour envoyer et récupérer des fichiers entre la machine locale et le serveur distant. Les commandes ont été exécutées avec succès, confirmant le bon fonctionnement de la connexion SFTP.

```
sftp> ls
mon_fichier.txt
sftp> get mon_fichier.txt
Fetching /home/sio/mon_fichier.txt to mon_fichier.txt
mon_fichier.txt                                       100%   17    18.7KB/s   00:00
sftp> put mon_fichier.txt
Uploading mon_fichier.txt to /home/sio/mon_fichier.txt
mon_fichier.txt                                       100%   17    20.3KB/s   00:00
sftp> exit
```

- `ls` → J'ai listé les fichiers présents sur le serveur. Le fichier `mon_fichier.txt` était bien présent, confirmant sa présence sur le serveur.

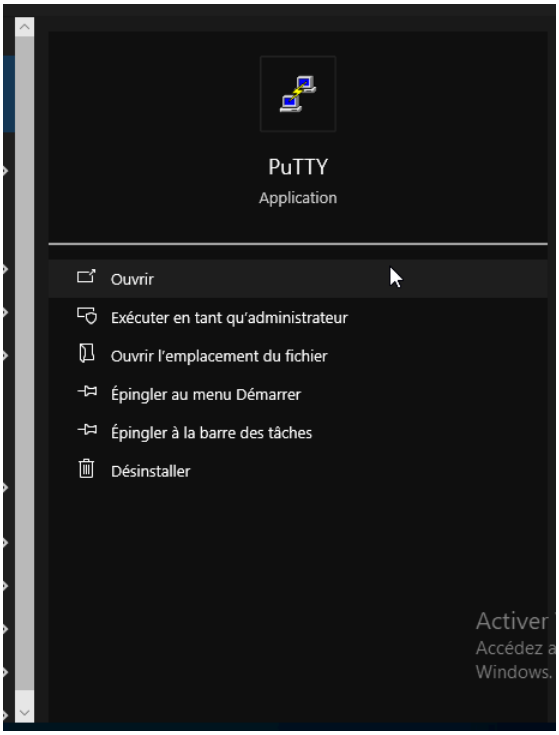
- `get mon_fichier.txt` → J'ai récupéré le fichier depuis le serveur vers ma VM cliente. Le fichier est désormais disponible localement.

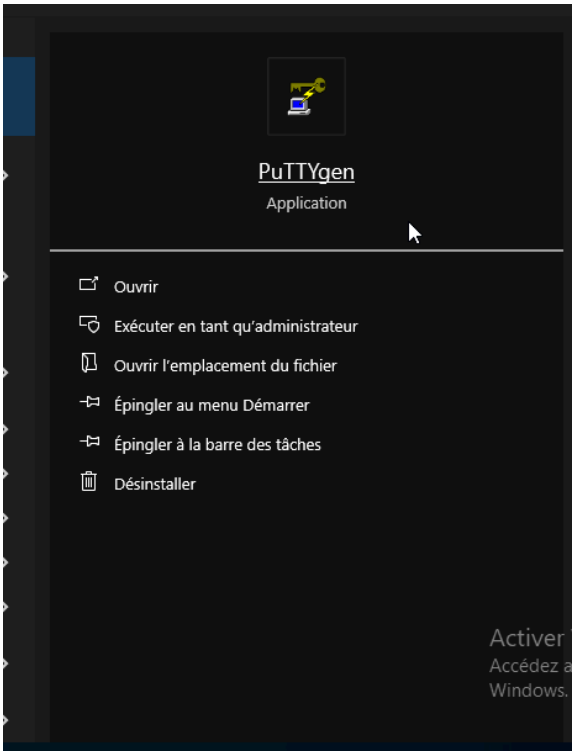
- `put mon_fichier.txt` → J'ai envoyé le fichier de ma VM cliente vers le serveur. Comme le fichier porte le même nom, il a été écrasé ou mis à jour sur le serveur.

- `exit` → J'ai quitté la session SFTP.

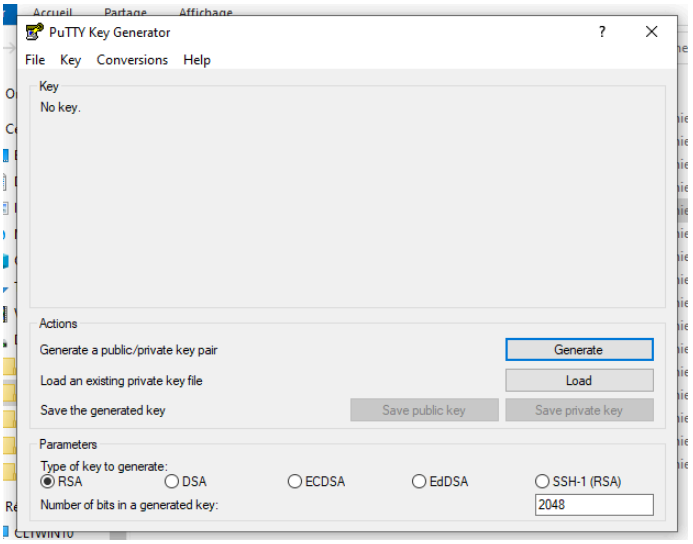
Installation des utilitaires Windows (PuTTY et PuTTYgen)

Sur une machine Windows 10, j'ai installé les utilitaires PuTTY et PuTTYgen. Ces outils permettent de se connecter en SSH et de générer des clés SSH pour sécuriser les connexions

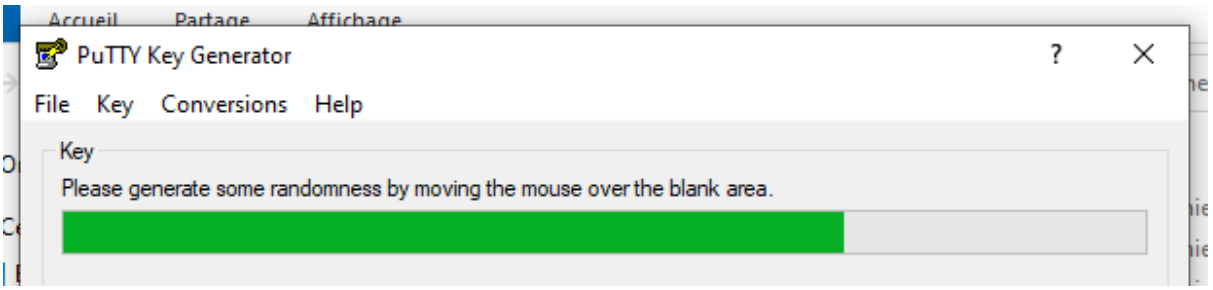




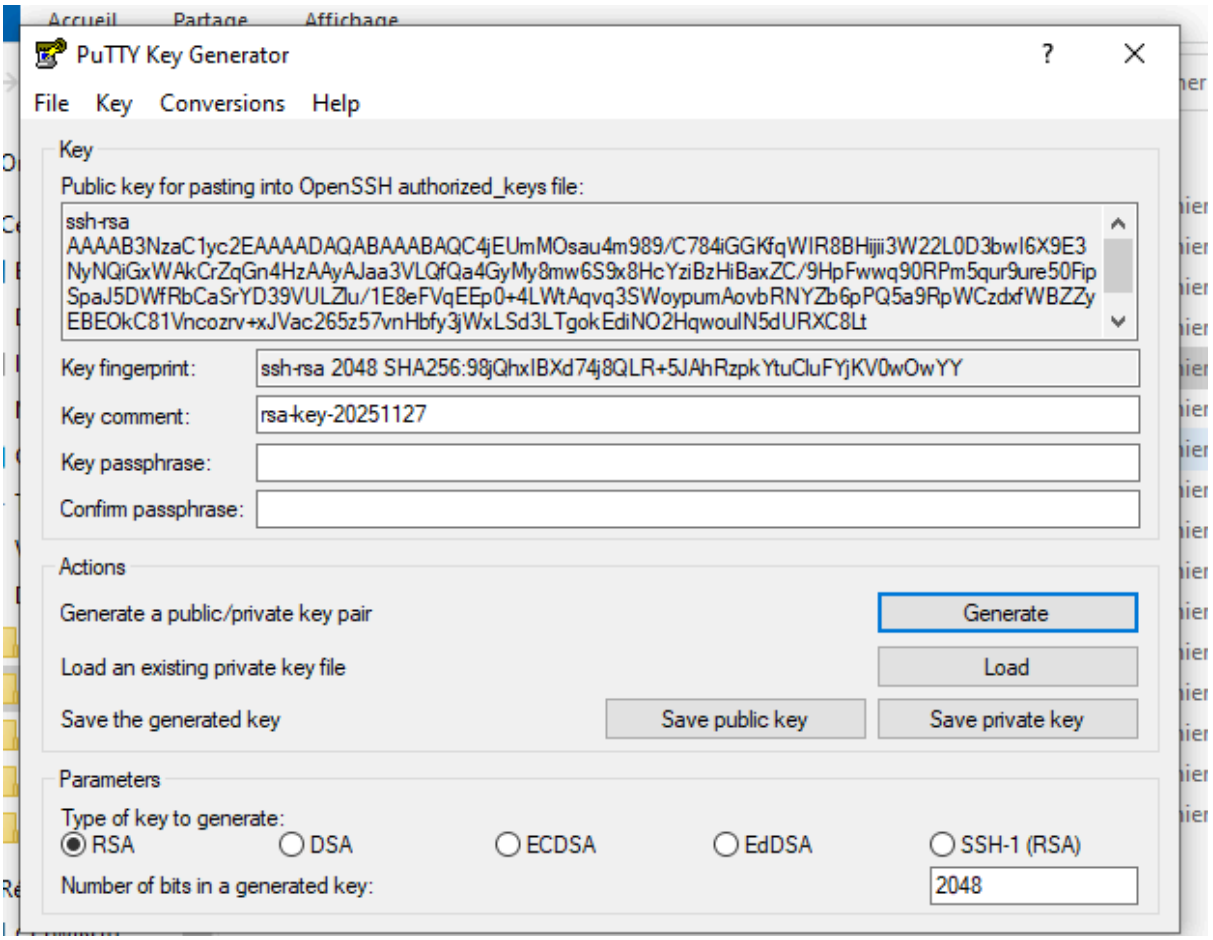
J'ai utilisé PuTTYgen pour générer une paire de clés SSH (clé publique et clé privée) sur Windows. Cette paire de clés permettra d'établir des connexions SSH sécurisées sans mot de passe.



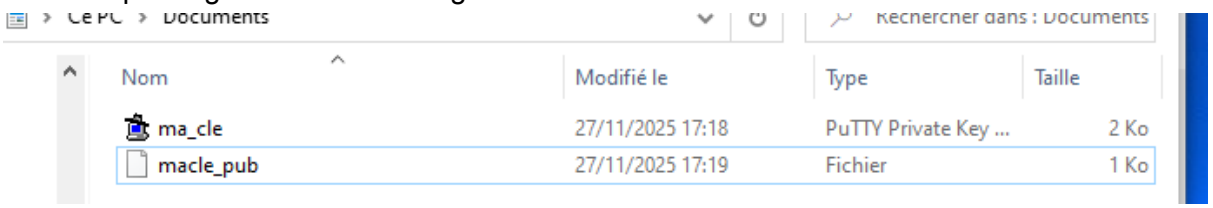
J'ai créé la clé en choisissant le type de clé souhaité (RSA) et en cliquant sur Generate dans PuTTYgen. La clé publique et la clé privée ont été générées, prêtes à être utilisées pour l'authentification SSH.



J'ai généré une nouvelle paire de clés SSH à partir de l'utilitaire PuTTYgen pour un utilisateur spécifique. La clé publique et la clé privée ont été créées et sont prêtes à être utilisées pour l'authentification SSH.



J'ai sauvegardé la clé privée en cliquant sur **Save private key** et en choisissant un emplacement sûr sur mon PC Windows. La sauvegarde de la clé publique est optionnelle, mais elle peut également être enregistrée si nécessaire.



J'ai commencé par créer le dossier `.ssh` sur le serveur Linux afin d'y ajouter la clé publique générée avec PuTTYgen. Cela permet de configurer l'authentification SSH par clé pour l'utilisateur ciblé.

```
sio@srv-ssh:~$ mkdir -p ~/.ssh
```

Pour finaliser, j'ai reproduit toute la procédure sur les serveurs concernés, à savoir **Nagios** et **GLPI**, afin de permettre une connexion SSH correcte à travers **cltlinux2**.

La capture d'écran illustre un test de connexion réalisé depuis le client vers le serveur, confirmant le bon fonctionnement de l'accès SSH.

```
root@cltlinux2:~# ssh -p 2222 sio@192.168.13.10
Linux glpi 6.12.48+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.48-1 (2025-09-20) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov 28 09:42:43 2025 from 192.168.13.45
sio@glpi:~$
```

Nous avons reproduit la même procédure pour le serveur Nagios, cette fois en utilisant un autre poste client, client-linux.

```
sio@Client-linux:~$ ssh -p 2222 sio@192.168.13.6
Linux Nagios1 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (2025-08-12) x86
_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov 28 09:54:40 2025 from 192.168.13.4
sio@Nagios1:~$
```

J'ai configuré la redirection des logs d'authentification SSH vers le serveur central Rsyslog dans un fichier dédié. Pour cela, j'ai créé un fichier de configuration spécifique au service SSH, permettant d'envoyer tous les logs générés par le programme `sshd` vers `/var/log/ssh.log`. La directive `& stop` a été utilisée afin d'empêcher que ces logs soient également enregistrés dans les fichiers de logs standards du système. Cette configuration assure une centralisation claire et isolée des logs SSH, facilitant ainsi leur analyse et le suivi de la sécurité.

```
192.168.13.5
192.168.13.45    192.168.13.5
if $programname == 'sshd' then /var/log/ssh.log
& stop
~
~
~
~
~
~
~
~
```

J'ai configuré SSH pour que ses logs d'authentification soient redirigés vers `/var/log/ssh.log` via Rsyslog, en utilisant `& stop` pour éviter qu'ils soient dupliqués dans les autres fichiers de logs.

```
^C
root@SrvRsyslog:~# mkdir -p /var/log/rsyslog/nagios
root@SrvRsyslog:~# touch /var/log/rsyslog/nagios/ssh.log
root@SrvRsyslog:~# chmod 640 /var/log/rsyslog/nagios/ssh.log
root@SrvRsyslog:~# chown root:adm /var/log/rsyslog/nagios/ssh.log
root@SrvRsyslog:~#
```

J'ai configuré SSH pour que ses logs d'authentification soient redirigés vers `/var/log/ssh.log` via Rsyslog, avec la directive `& stop` afin d'éviter leur duplication dans les autres fichiers de logs.

```
~
~
~
~
~
"/etc/rsyslog.d/50-local-ssh.conf" [Nouveau]                    0,0-1        Tout
```

On y ajoute ceci

```
192.168.13.45    192.168.13.5
# Charger le module imfile
module(load="imfile")

# Surveiller le fichier tmp_id_rsa.pub et envoyer vers Rsyslog central
$InputFileName /home/sio/tmp_id_rsa.pub
$InputFileTag ssh-file:
$InputFileStateFile stat-ssh-file
$InputRunFileMonitor

# Envoyer toutes les entrées vers le serveur Rsyslog central
*.*    @192.168.13.5:514
```

Avant de commencer sur GLPI, j'ai créé un répertoire `rsyslog.d`. Ce répertoire me permet de stocker des fichiers de configuration spécifiques pour le service rsyslog, qui centralise et gère les journaux système sur Linux. Cela facilite la personnalisation et la gestion des logs pour notre serveur.

```
192.168.13.45    192.168.13.5    192.168.13.5
GNU nano 8.4                    /etc/rsyslog.d/50-local-ssh.conf *
# Stocker les logs SSH envoyés par GLPI
$template NagiosSSHLog,"/var/log/rsyslog/nagios/ssh.log"
if $programname == 'ssh-file:' then -?NagiosSSHLog
& stop
```

Je n'ai pas finis.

Mission 2: VPN

1] Installation et configuration d'openvpn et de la PKI

J'ai installé les paquets `openvpn` en utilisant l'option `-y` pour automatiser la confirmation des installations. OpenVPN est un logiciel qui permet de créer des réseaux privés virtuels (VPN), assurant une connexion sécurisée entre différents postes ou serveurs.

```
root@Srv-vpn:~# apt install openvpn -y
openvpn est déjà la version la plus récente (2.6.14-1).
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libcbor0.8          libngtcp2-16              libpython3.11-stdlib  python3-jaraco.context    python3-pyparsing          python3.11
  libcurl3t64-gnutls  libngtcp2-crypto-gnutls8  perl-modules-5.36     python3-jaraco.functools  python3-pysimplesoap       python3.11-minimal
  libfuse2            libnsl2                   python3-autocommand   python3-more-itertools    python3-six
  libicu72            libperl5.36               python3-httplib2      python3-pkg-resources     python3-typeguard
  libldap-2.5-0       libpython3.11-minimal     python3-inflect       python3-pycurl            python3-typing-extensions
Veuillez utiliser « apt autoremove » pour les supprimer.

Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 53
root@Srv-vpn:~# _
```

J'ai installé les paquets `openssl` en utilisant l'option `-y` pour automatiser la confirmation des installations. OpenSSL est une bibliothèque qui permet de gérer la cryptographie, notamment la création de certificats, le chiffrement et le déchiffrement des données, ce qui est essentiel pour sécuriser les communications réseau.

```
root@Srv-vpn:~# apt install openssl -y
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libcbor0.8          libngtcp2-16              libpython3.11-stdlib  python3-jaraco.context    python3-pyparsing          python3.11
  libcurl3t64-gnutls  libngtcp2-crypto-gnutls8  perl-modules-5.36     python3-jaraco.functools  python3-pysimplesoap       python3.11-minima
  libfuse2            libnsl2                   python3-autocommand   python3-more-itertools    python3-six
  libicu72            libperl5.36               python3-httplib2      python3-pkg-resources     python3-typeguard
  libldap-2.5-0       libpython3.11-minimal     python3-inflect       python3-pycurl            python3-typing-extensions
Veuillez utiliser « apt autoremove » pour les supprimer.

Mis à jour :
  libssl3t64  openssl  openssl-provider-legacy

Sommaire :
  Mise à niveau de : 3. Installation de : 0Supprimé : 0. Non mis à jour : 50
Taille du téléchargement : 4 244 kB
  Espace nécessaire : 8 192 B / 28,5 GB disponible

Réception de : 1 http://deb.debian.org/debian trixie/main amd64 openssl-provider-legacy amd64 3.5.4-1~deb13u1 [308 kB]
Réception de : 2 http://deb.debian.org/debian trixie/main amd64 libssl3t64 amd64 3.5.4-1~deb13u1 [2 441 kB]
Réception de : 3 http://deb.debian.org/debian trixie/main amd64 openssl amd64 3.5.4-1~deb13u1 [1 495 kB]
4 244 ko réceptionnés en 0s (16,5 Mo/s)
apt-listchanges : Lecture des fichiers de modifications (« changelog »)...
(Lecture de la base de données... 44780 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../openssl-provider-legacy_3.5.4-1~deb13u1_amd64.deb ...
Dépaquetage de openssl-provider-legacy (3.5.4-1~deb13u1) sur (3.5.1-1) ...
Paramétrage de openssl-provider-legacy (3.5.4-1~deb13u1) ...
(Lecture de la base de données... 44780 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../libssl3t64_3.5.4-1~deb13u1_amd64.deb ...
Dépaquetage de libssl3t64:amd64 (3.5.4-1~deb13u1) sur (3.5.1-1) ...

Progression : [ 38%] [
```

J'ai vérifié la version d'OpenVPN installée sur le serveur afin de m'assurer que le logiciel est à jour et compatible avec notre configuration VPN.

```
root@Srv-vpn:~# openvpn --version
OpenVPN 2.6.14 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO
]
library versions: OpenSSL 3.5.4 30 Sep 2025, LZO 2.10
DCO version: N/A
Originally developed by James Yonan
Copyright (C) 2002-2024 OpenVPN Inc <sales@openvpn.net>
Compile time defines: enable_async_push=no enable_comp_stub=no enable_crypto_ofb_cfb=yes enable_dco=yes
enable_dco_arg=yes enable_debug=yes enable_dependency_tracking=no enable_dlopen=unknown enable_dlopen_se
lf=unknown enable_dlopen_self_static=unknown enable_fast_install=needless enable_fragment=yes enable_ipr
oute2=no enable_libtool_lock=yes enable_lz4=yes enable_lzo=yes enable_maintainer_mode=no enable_manageme
nt=yes enable_option_checking=no enable_pam_dlopen=no enable_pedantic=no enable_pkcs11=yes enable_plugin
_auth_pam=yes enable_plugin_down_root=yes enable_plugins=yes enable_port_share=yes enable_selinux=no ena
ble_shared=yes enable_shared_with_static_runtimes=no enable_silent_rules=no enable_small=no enable_stati
c=yes enable_strict=no enable_strict_options=no enable_systemd=yes enable_unit_tests=no enable_werror=no
 enable_win32_dll=yes enable_wolfssl_options_h=yes enable_x509_alt_username=yes with_aix_soname=aix with
_crypto_library=openssl with_gnu_ld=yes with_mem_check=no with_openssl_engine=auto with_sysroot=no
root@Srv-vpn:~#
```

J'ai créé le répertoire **Easy-RSA** ainsi que le répertoire de configuration **OpenVPN**.

- Le répertoire `/etc/openvpn/server/` contient tous les fichiers nécessaires au fonctionnement du serveur VPN.

- Le répertoire `/etc/openvpn/client/` sera utilisé pour stocker les fichiers destinés aux futurs clients VPN.

```
root@Srv-vpn:~# ls /usr/share/easy-rsa/
easyrsa  openssl-easyrsa.cnf  vars.example  x509-types
root@Srv-vpn:~# ls /etc/openvpn/
client  easy-rsa  server  update-resolv-conf
```

J'ai suivi la bonne pratique qui consiste à travailler avec Easy-RSA dans le répertoire `/etc/openvpn/easy-rsa/`. Cela permet de centraliser et d'organiser la création et la gestion des certificats pour le VPN, facilitant ainsi la maintenance et la sécurité du serveur.

```
root@Srv-vpn:~# cp -r /usr/share/easy-rsa/ /etc/openvpn/
root@Srv-vpn:~# ls /etc/openvpn/easy-rsa/
easyrsa  openssl-easyrsa.cnf  pki  vars  vars.example  x509-types
root@Srv-vpn:~#
```

Easy-RSA m'a demandé de confirmer que je souhaitais supprimer toute **PKI** existante dans /etc/openvpn/easy-rsa/pki et créer une nouvelle PKI. J'ai validé, et ma PKI a été initialisée avec succès. Le répertoire /etc/openvpn/easy-rsa/pki a été créé pour stocker les certificats et clés cryptographiques nécessaires au fonctionnement sécurisé du VPN

```
root@Srv-vpn:/etc/openvpn/easy-rsa# ./easyrsa init-pki
Using Easy-RSA 'vars' configuration:
* /etc/openvpn/easy-rsa/vars

WARNING!!!

You are about to remove the EASYRSA_PKI at:
* /etc/openvpn/easy-rsa/pki

and initialize a fresh PKI here.

Type the word 'yes' to continue, or any other input to abort.
  Confirm removal: yes

         ******************************************
         * SECOND WARNING - STOP - SECOND WARNING *
         ******************************************

  To keep your current 'pki/vars' settings use 'init-pki soft'.
  To keep your current Request files use 'init-pki soft'
  The Requests can then be signed by a new CA (Partial CA renewal)
  To keep your current Easy-RSA TLS Key use 'init-pki soft'
  This private key file is in use by your current VPN.

       ** USE OF   'init-pki soft'   IS RECOMMENDED **


Type the word 'yes' to continue, or any other input to abort.

  WARNING: COMPLETELY DESTROY current PKI (NOT recommended) ?

    [yes/NO]: yes


Notice
------
'init-pki' complete; you may now create a CA or requests.

Your newly created PKI dir is:
* /etc/openvpn/easy-rsa/pki

Using Easy-RSA configuration:
* /etc/openvpn/easy-rsa/vars
```

on remplie le passphrase plus on rempli et le sio

```
Enter New CA Key Passphrase:

Confirm New CA Key Passphrase:
.....+.....+...++++++++++++++++++++++++++++++++++++++++*......+......+..+...+....+.........+.....+.+.....
+......+.......++++++++++++++++++++++++++++++++++++++++*......+.........+.+...+........+....+...+........
+...+....+...+..+.......+.....+.........+.+......+.....+....+.........+.....+......+......+.........+...
..................+......................+..+...+......+.......+........+...............++++++
.+...........+.+......+..............+......+....+...+..+....+.........+...+..+..........+..+.++++++++++
+++++++++++++++++++++++++++++++*.+................+...........+...+.+......+.....+...+...+....+......+++++
++++++++++++++++++++++++++++++++++*.....+.....+....+..+...+.......+...+......+.....+.+........+.........
...+.........+.+.........+...........+.+.....+....+.....+.+.....+.......+......+.........+..+...+.+.....
............+...+....+......+.....+.........+...+.......+...+........+....+..+...............+.+...+....
..........+.......+......+..+.+..+....+...+............+.....+...+...+...+.+......+....................+
...+....+...........+.........+..+....+..........+....+...+...+..+............+....+...........+.+....
....+......+....+.........+..+.+..+..........+......+..+...+.+...............++++++
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Common Name (eg: your user, host, or server name) [Easy-RSA CA]:sio

Notice
------
CA creation complete. Your new CA certificate is at:
* /etc/openvpn/easy-rsa/pki/ca.crt

Create an OpenVPN TLS-AUTH|TLS-CRYPT-V1 key now: See 'help gen-tls'

Build-ca completed successfully.
```

J'ai généré la clé privée et la requête de signature (CSR) pour mon serveur VPN.

- La clé privée du serveur est stockée dans /etc/openvpn/easy-rsa/pki/private/srvVPN.key et est essentielle pour sécuriser les communications du serveur.

- La requête de certificat (CSR) se trouve dans /etc/openvpn/easy-rsa/pki/reqs/srvVPN.req et servira à obtenir un certificat signé permettant l'authentification du serveur auprès des clients VPN.

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Common Name (eg: your user, host, or server name) [srvVPN]:sio

Notice
------
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /etc/openvpn/easy-rsa/pki/reqs/srvVPN.req
* key: /etc/openvpn/easy-rsa/pki/private/srvVPN.key
```

J'ai copié les fichiers nécessaires au bon fonctionnement du serveur OpenVPN. Cette étape est importante pour que le serveur dispose de toutes les clés, certificats et fichiers de configuration requis afin d'établir des connexions VPN sécurisées avec les clients.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# mkdir -p /etc/openvpn/server
cp pki/ca.crt pki/issued/srvVPN.crt pki/private/srvVPN.key /etc/openvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

Je me suis placé dans le répertoire /etc/openvpn/easy-rsa et j'ai généré une clé Diffie-Hellman avec la commande ./easyrsa gen-dh. Cette clé est essentielle pour permettre un échange sécurisé des clés entre le serveur et les clients, garantissant ainsi la confidentialité et la sécurité des communications VPN.

```
*++*++*
DH parameters appear to be ok.

Notice
------

DH parameters of size 2048 created at:
* /etc/openvpn/easy-rsa/pki/dh.pem

root@Srv-vpn:/etc/openvpn/easy-rsa# ./easyrsa gen-dh
```

J'ai copié les fichiers nécessaires vers le répertoire du serveur OpenVPN afin que celui-ci dispose de toutes les clés, certificats et fichiers de configuration indispensables pour établir des connexions VPN sécurisées avec les clients.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# cp pki/dh.pem /etc/openvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

J'ai créé le fichier `vars`, qui servira de configuration pour le script **Easy-RSA**. Ce fichier contient des variables importantes, comme le nom de l'organisation, le pays ou encore les chemins des répertoires, permettant d'automatiser et de standardiser la génération des certificats et des clés pour le VPN.

```
192.168.13.45   192.168.13.5   192.168.13.5   192.168.13.18   192.168.13.4
  GNU nano 8.4                    /etc/openvpn/easy-rsa/pki/vars
# Paramètres cryptographiques (ne pas modifier)
set_var EASYRSA_ALGO            "rsa"
set_var EASYRSA_DIGEST          "sha256"
set_var EASYRSA_KEY_SIZE        2048

# Informations pour les certificats (adapté pour toi)
set_var EASYRSA_REQ_COUNTRY     "FR"
set_var EASYRSA_REQ_PROVINCE    "IDF"
set_var EASYRSA_REQ_CITY        "Melun"
set_var EASYRSA_REQ_ORG         "Lycee Leonard de Vinci"
set_var EASYRSA_REQ_EMAIL       "admin@lycee-leonard-vinci.fr"
set_var EASYRSA_REQ_OU          "SISR"

# Nom du certificat de l'autorité (CA)
set_var EASYRSA_REQ_CN          "srv-vpn"

# Chemin où seront stockés les fichiers
set_var EASYRSA_PKI             "/etc/openvpn/easy-rsa/pki"

# Option de sécurité (laisser)
set_var EASYRSA_SAFE_CONF       "yes"

set_var EASYRSA_PKI              "org"
```

création du certificat qui est bien céer.

```
* /etc/openvpn/easy-rsa/vars
root@Srv-vpn:/etc/openvpn/easy-rsa# ./easyrsa build-ca
Using Easy-RSA 'vars' configuration:
* /etc/openvpn/easy-rsa/vars

Enter New CA Key Passphrase:

Confirm New CA Key Passphrase:
.............+.....+.+...+......+.....+++++++++++++++++++++++++++++++++++++++++*..+++++++++++
++++++++++++++++++++++++++++*....+...+........+....+......+.....+...+.+.....+..............
.....+......+.....+.+............+......+.........+..+.+.....+.........+...+...+....+...+..
...+.+......+........+...............+...+...+.+.....+.+.........+...........+.............
...............+.....+...............+..........+......+........+.............+.....+......
.+......+..+...+....++++++
.....+....+.....+.+.....+.+...+++++++++++++++++++++++++++++++++++++++++*...+...+...+.....+...
....+...+++++++++++++++++++++++++++++++++++++++*.+.......................+....+............
+.....+.+...+..............+.+...............+..+...+..........+......+...+............+...
.....+...+...+.+...+...+.....+......................+........+....+.........+.....+.......+
...+..+....+......+........+..........+.........+..+.........+......+......+..........+++++
+
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Common Name (eg: your user, host, or server name) [Easy-RSA CA]:srv-vpn

Notice
------
CA creation complete. Your new CA certificate is at:
* /etc/openvpn/easy-rsa/pki/ca.crt

Create an OpenVPN TLS-AUTH|TLS-CRYPT-V1 key now: See 'help gen-tls'

Build-ca completed successfully.
```

2] Génération du certificat et de la clé d'autorité de certification

J'ai généré la **demande de certificat (CSR)** pour le serveur VPN. Cette demande contient les informations nécessaires à l'émission d'un certificat, qui permettra au serveur d'être authentifié et reconnu comme fiable par les clients VPN.

```
  Requested CN:     'srv-vpn'
  Requested type:   'server'
  Valid for:        '365' days


subject=
    commonName                = srv-vpn

Type the word 'yes' to continue, or any other input to abort.
  Confirm requested details:


Notice
------
Aborting without confirmation.
```

J'ai créé le **certificat du serveur VPN** avec succès. Ce certificat permet d'authentifier le serveur auprès des clients VPN et de sécuriser les échanges de données.

```
Certificate created at:
* /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt
```

Après la **signature du certificat**, j'ai vérifié que le fichier existait bien. Cette vérification permet de s'assurer que le certificat a été correctement généré et est prêt à être utilisé par le serveur VPN.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# ls -l /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt
-rw------- 1 root root 4492 28 nov.  15:43 /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt
```

J'ai consulté le **contenu du certificat** pour vérifier ses informations, telles que le nom du serveur, la période de validité et les détails de l'autorité de certification. Cette étape permet de confirmer que le certificat est correct et utilisable pour sécuriser le VPN.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# openssl x509 -in /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            66:98:90:7a:ae:18:4c:1b:ec:1d:07:30:02:67:dd:a9
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN=srv-vpn
        Validity
            Not Before: Nov 28 14:43:43 2025 GMT
            Not After : Nov 28 14:43:43 2026 GMT
        Subject: CN=srv-vpn
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
```

J'ai vérifié que le certificat était bien **signé par l'autorité de certification (CA)**. Cette vérification est essentielle pour confirmer que le certificat est authentique et reconnu par le serveur et les clients VPN.

```
        42:76:2a:98
root@Srv-vpn:/etc/openvpn/easy-rsa# openssl verify -CAfile /etc/openvpn/easy-rsa/pki/ca.crt
 /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt
/etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt: OK
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

3] Génération du certificat et de la clé pour le serveur VPN

Je me suis placé dans le répertoire `/etc/openvpn/easy-rsa` et j'ai généré la clé privée ainsi que la demande de certificat (CSR) pour mon serveur VPN en utilisant la commande `./easyrsa gen-req SrvVPN nopass`. Le nom `SrvVPN` correspond au nom du serveur (Common Name / CN) et l'option `nopass` permet de ne pas mettre de mot de passe sur la clé, ce qui facilite le démarrage automatique du serveur. Cette opération a créé deux fichiers importants : la clé privée `pki/private/SrvVPN.key`, que je dois garder strictement confidentielle, et la demande de certificat `pki/reqs/SrvVPN.req`, qui sera ensuite signée par l'autorité de certification (CA) pour authentifier le serveur VPN.

```
Notice
------
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /etc/openvpn/easy-rsa/pki/reqs/SrvVPN.req
* key: /etc/openvpn/easy-rsa/pki/private/SrvVPN.key
```

J'ai signé le certificat du serveur avec l'autorité de certification (CA) en utilisant la commande ./easyrsa sign-req server SrvVPN. Au préalable, à partir du script Easy-RSA, j'avais généré la clé privée du serveur (fichier .key) ainsi que la demande de signature de certificat (fichier .req). La signature par la CA permet d'authentifier le serveur VPN et de sécuriser les échanges avec les clients.

```
Notice
------
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /etc/openvpn/easy-rsa/pki/reqs/SrvVPN.req
* key: /etc/openvpn/easy-rsa/pki/private/SrvVPN.key
```

J'ai distingué la clé privée et la clé publique de mon serveur VPN. La clé privée, située dans `/etc/openvpn/easy-rsa/pki/private/SrvVPN.key`, est strictement confidentielle et ne doit jamais être partagée. Le certificat du serveur, quant à lui, se trouve dans `/etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt` et contient la clé publique ainsi que toutes les informations signées par l'autorité de certification (CA). Ainsi, la clé publique n'existe pas comme fichier séparé, elle est intégrée directement dans le certificat SrvVPN.crt. J'ai pu consulter la clé publique RSA du serveur directement dans le terminal à partir de ce certificat.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# openssl x509 -in /etc/openvpn/easy-rsa/pki/issued/SrvVP
N.crt -pubkey -noout
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAjX056SrJqTTg6vwQbgvB
Y51ckjEzh9Go9gwdNz4nEzD9yBJxJsxUxsI+zDKAu4Z63KQtBi8rJVWDNiaNuEAg
zWQfVNPlqSzdyKCKOHLE1DLiUlSsRivhGD2GSRmCJKGEBEJemI0g8CSoJj6jjtCo
7AAKBwoJbRJwfYH6xNpS1PdHsznuVo4nvI4EW/TVoHXIpaGWx+o0NGq3cq7Dpp21
fWzIDWli6sepRO/f6yctTySn2tLDKe/0Jj/KJCoUDI0ErjXBU2d1DRGcCy75t9Dj
+kvmzinAMcWih1C+Va4VRRI2unvbGaMKOw5Z2YvGtRculYzLDiMPv7IuM2KeAtv0
zQIDAQAB
-----END PUBLIC KEY-----
```

J’ai vérifié la validité du certificat du serveur en utilisant une commande OpenSSL. Cette vérification m’a permis de m’assurer que le certificat est correct, authentique et prêt à sécuriser les communications VPN entre le serveur et les clients.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# openssl verify -CAfile pki/ca.crt pki/issued/SrvVPN.crt
pki/issued/SrvVPN.crt: OK
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

5] Génération des paramètres de Diffie-Hellman

Toujours avec le script Easy-RSA, j'ai généré les paramètres Diffie-Hellman, essentiels pour sécuriser l'échange des clés entre le serveur et les clients VPN. Cette opération a créé le fichier `/etc/openvpn/easy-rsa/pki/dh.pem`, qui sera utilisé par le serveur pour établir des connexions chiffrées et sécurisées.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# ./easyrsa gen-dh
Using Easy-RSA 'vars' configuration:
* /etc/openvpn/easy-rsa/vars
Generating DH parameters, 2048 bit long safe prime
.....................................................................................
.............................+.......................................................
.....................................................................................
.........................+...............................+.............................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.................................+...................................................
.............+.......................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....+...................................+...................................+.........
.......+.............................................................................
.....................................................................................
.................................................+...........+.........................+........
....................+................................................................
.....................................................................................
.....................................................................................
...............................+.....................................................+...............
.....................................................................................
.........................................++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++
*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*
++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*
DH parameters appear to be ok.

Notice
------

DH parameters of size 2048 created at:
* /etc/openvpn/easy-rsa/pki/dh.pem
```

Le fichier `dh.pem` contient les paramètres Diffie-Hellman utilisés par OpenVPN pour négocier une clé symétrique sécurisée entre le serveur et les clients. Ces paramètres permettent de générer une clé commune sans jamais l'échanger directement, assurant ainsi la confidentialité des communications. Je n'ai pas besoin de taper `dh` dans le terminal, car le fichier est simplement utilisé dans la configuration du serveur.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# cat /etc/openvpn/easy-rsa/pki/dh.pem
-----BEGIN DH PARAMETERS-----
MIIBDAKCAQEA/yCKkUwYX+H7MFjyLA4BczG81jF6tk2iJAsr9luaBM8GTVKqSU3A
FQNUsKreN2E1VcNzMDiQiwgs3oryqAxxopq97Fcmw553b1UnMN+FBHpv6+g7/7PC
3GRelev3gnJMKnYYw+vEo7HgOMWxlvPJ247x0X4+evcqc7rvaqHqushx5Dw/0Z7V
HG9AuDwb3QkPp7SOvyQDpVPIOg8XBn6J4DQuUBAgAeYfFVcS2WnyY02L2xHVkbUl
0pFlAxnb1MQufZGCEWWkj4Ctv2BVcBJgXdA74/YeP0JlAvGWIeelN9+IrND7OUNo
mlFWsT9h4LxzKwwHROvWZG6xLA0OCOhj/wIBAgICAOE=
-----END DH PARAMETERS-----
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

6] Répartition des clés entre client et serveur

J'ai positionné les différents fichiers nécessaires au serveur VPN dans le répertoire approprié `/etc/openvpn/server/`. Les fichiers incluent : le certificat de l'autorité de certification `ca.crt` (non secret), la clé privée de l'autorité de certification `ca.key` (secret), les paramètres Diffie-Hellman `dh.pem` (non secret), le certificat du serveur `NomServeur.crt` (non secret) et la clé privée du serveur `NomServeur.key` (secret). Placer correctement ces fichiers est essentiel pour que le serveur OpenVPN fonctionne et puisse sécuriser les connexions avec les clients.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# cp pki/ca.crt /etc/openvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

J'ai copié les fichiers dans `/etc/openvpn/server/` pour que le serveur dispose de sa clé privée, de son certificat, du certificat de l'autorité de certification et des paramètres Diffie-Hellman, lui permettant ainsi de s'authentifier et de sécuriser les connexions VPN avec les clients.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# cp pki/ca.crt /etc/openvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa# mv /etc/openvpn/easy-rsa/pki/private/SrvVPN.key /etc/op
envpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa# mv /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt /etc/ope
nvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa# mv /etc/openvpn/easy-rsa/pki/dh.pem /etc/openvpn/server
/
```

J'ai créé le certificat signé du client, situé dans `/etc/openvpn/easy-rsa/pki/issued/NomClient.crt`. Ce certificat contient la clé publique du client, signée par l'autorité de certification (CA), ce qui permet au serveur de vérifier l'identité du client. J'ai également généré un fichier inline dans `/etc/openvpn/easy-rsa/pki/inline/private/NomClient.inline`, qui regroupe la clé privée du client et son certificat dans un seul fichier. Ce fichier est secret et ne doit jamais être partagé, mais il est pratique pour certaines configurations où un fichier unique est nécessaire pour la connexion du client au VPN.

```
Notice
------
Inline file created:
* /etc/openvpn/easy-rsa/pki/inline/private/NomClient.inline


Notice
------
Certificate created at:
* /etc/openvpn/easy-rsa/pki/issued/NomClient.crt
```

J'ai créé la PKI et l'autorité de certification (CA) avec Easy-RSA en exécutant `./easyrsa init-pki` puis `./easyrsa build-ca nopass`, afin de pouvoir signer les certificats du serveur et des clients pour sécuriser le VPN.

```
Notice
------
CA creation complete. Your new CA certificate is at:
* /etc/openvpn/easy-rsa/pki/ca.crt

Create an OpenVPN TLS-AUTH|TLS-CRYPT-V1 key now: See 'help gen-tls'

Build-ca completed successfully.

root@Srv-vpn:/etc/openvpn/easy-rsa#
```

J'ai généré la clé privée et le certificat du serveur en utilisant Easy-RSA avec les commandes `./easyrsa gen-req SrvVPN nopass` puis `./easyrsa sign-req server SrvVPN`. Cette opération a créé les fichiers `SrvVPN.key` et `SrvVPN.crt`, nécessaires pour que le serveur s'authentifie et sécurise les connexions VPN.

```
Notice
------
Inline file created:
* /etc/openvpn/easy-rsa/pki/inline/private/SrvVPN.inline


Notice
------
Certificate created at:
* /etc/openvpn/easy-rsa/pki/issued/SrvVPN.crt
```

J'ai généré les paramètres Diffie-Hellman avec la commande `./easyrsa gen-dh`. Ces paramètres sont essentiels pour permettre un échange sécurisé des clés entre le serveur et les clients VPN.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# ./easyrsa gen-dh
Using Easy-RSA 'vars' configuration:
* /etc/openvpn/easy-rsa/vars
Generating DH parameters, 2048 bit long safe prime
.....................................................+.................+...........
.....................................................................................
.....................................................................................
.....................................................................................
+....................................................................................
.....................................................................................
...................................................+.................................
.....................................................................................
.++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*
++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*++*+
+*++*++*++*
DH parameters appear to be ok.

Notice
------

DH parameters of size 2048 created at:
* /etc/openvpn/easy-rsa/pki/dh.pem

root@Srv-vpn:/etc/openvpn/easy-rsa#
```

J'ai placé les fichiers nécessaires dans le répertoire du serveur `/etc/openvpn/server/` en copiant `ca.crt` et en déplaçant `SrvVPN.key`, `SrvVPN.crt` et `dh.pem`. Le répertoire contient désormais tous les fichiers indispensables au fonctionnement du serveur. Seules la **clé privée du serveur** (`SrvVPN.key`) et la **clé de la CA** (`ca.key`) restent confidentielles.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# cp pki/ca.crt /etc/openvpn/server/
mv pki/private/SrvVPN.key /etc/openvpn/server/
mv pki/issued/SrvVPN.crt /etc/openvpn/server/
mv pki/dh.pem /etc/openvpn/server/
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

8] Configuration et démarrage du serveur VPN

J'ai décidé de partir d'un fichier vierge pour la configuration du serveur VPN. À l'aide de l'éditeur vim, j'ai créé et édité le fichier `/etc/openvpn/server.conf`. Dans ce fichier, j'ai inséré les directives nécessaires, en tenant compte que toute ligne débutant par un « # » est un commentaire. Les directives à inclure ou à adapter ont été soigneusement configurées pour assurer le bon fonctionnement du serveur.

```
192.168.13.45   192.168.13.5   192.168.13.5   192.168.13.18
  GNU nano 8.4                /etc/openvpn/server/server.conf *
mode server
tls-server
local 192.168.13.18
port 1194
proto udp
dev tun

ca /etc/openvpn/server/ca.crt
cert /etc/openvpn/server/SrvVPN.crt
key /etc/openvpn/server/SrvVPN.key
dh /etc/openvpn/server/dh.pem

server 10.8.0.0 255.255.255.0
push "route 10.8.0.1 255.255.255.255"
push "redirect-gateway def1"

keepalive 10 120
user openvpn
group openvpn
persist-key
persist-tun
client-to-client
max-clients 10

# TLS Auth (si ta.key existe)
tls-auth /etc/openvpn/server/ta.key
key-direction 0

# Chroot (sécurité)
chroot /etc/openvpn/jail

log /var/log/openvpn/openvpn.log
status /var/log/openvpn/openvpn-status.log
verb 3
mute 20

tls-version-min 1.2
cipher AES-256-GCM
tls-cipher TLS-DHE-RSA-WITH-AES-256-GCM-SHA384
auth SHA256
```

J'ai redémarré le serveur OpenVPN en utilisant la commande classique, afin que les nouvelles configurations soient prises en compte et que le service VPN fonctionne correctement.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# systemctl restart openvpn
root@Srv-vpn:/etc/openvpn/easy-rsa# systemctl status openvpn
● openvpn.service - OpenVPN service
     Loaded: loaded (/usr/lib/systemd/system/openvpn.service; enabled; preset: enabled)
     Active: active (exited) since Fri 2025-11-28 16:31:36 CET; 5s ago
 Invocation: a8267cc05d81449f96b305f6bb39f02a
       Docs: man:openvpn(8)
    Process: 4611 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 4611 (code=exited, status=0/SUCCESS)
   Mem peak: 1.7M
        CPU: 10ms

nov. 28 16:31:36 Srv-vpn systemd[1]: Starting openvpn.service - OpenVPN service...
nov. 28 16:31:36 Srv-vpn systemd[1]: Finished openvpn.service - OpenVPN service.
root@Srv-vpn:/etc/openvpn/easy-rsa#
```

J’ai consulté les logs du serveur OpenVPN pour vérifier son fonctionnement. Les logs montrent que le service démarre puis s’arrête immédiatement, sans erreur fatale. Cela correspond au comportement attendu du service OpenVPN sur certaines distributions, où l’instance spécifique (`openvpn@server`) ne se lance pas automatiquement.

```
root@Srv-vpn:/etc/openvpn/easy-rsa# journalctl -f -u openvpn
nov. 28 10:43:52 Srv-vpn systemd[1]: Starting openvpn.service - OpenVPN service...
nov. 28 10:43:52 Srv-vpn systemd[1]: Finished openvpn.service - OpenVPN service.
nov. 28 16:31:36 Srv-vpn systemd[1]: openvpn.service: Deactivated successfully.
nov. 28 16:31:36 Srv-vpn systemd[1]: Stopped openvpn.service - OpenVPN service.
nov. 28 16:31:36 Srv-vpn systemd[1]: Stopping openvpn.service - OpenVPN service...
nov. 28 16:31:36 Srv-vpn systemd[1]: Starting openvpn.service - OpenVPN service...
nov. 28 16:31:36 Srv-vpn systemd[1]: Finished openvpn.service - OpenVPN service.
```

8] Le « forward » de paquets sur le serveur VPN

Pour que le serveur VPN fonctionne comme une passerelle entre le VPN et le réseau local, j'ai activé temporairement le forwarding IPv4 en exécutant la commande `echo 1 > /proc/sys/net/ipv4/ip_forward`, ce qui permet le transfert des paquets entre les interfaces réseau. Pour rendre cette configuration permanente, j'ai créé ou modifié le fichier `/etc/sysctl.d/sysctl.conf` et ajouté la ligne correspondante pour activer le forwarding à chaque démarrage du système.

La ligne ci-dessus

```
  GNU nano 8.4                    /etc/sysctl.d/sysctl.conf *
net.ipv4.ip_forward=1
```

Puis on applique les changements :

```
root@Srv-vpn:/etc/openvpn/easy-rsa# sysctl --system
* Applique /usr/lib/sysctl.d/10-coredump-debian.conf …
* Applique /usr/lib/sysctl.d/50-default.conf …
* Applique /usr/lib/sysctl.d/50-pid-max.conf …
* Applique /etc/sysctl.d/sysctl.conf …
kernel.core_pattern = core
kernel.sysrq = 0x01b6
kernel.core_uses_pid = 1
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.ens18.rp_filter = 2
net.ipv4.conf.lo.rp_filter = 2
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.ens18.accept_source_route = 0
net.ipv4.conf.lo.accept_source_route = 0
net.ipv4.conf.default.promote_secondaries = 1
net.ipv4.conf.ens18.promote_secondaries = 1
net.ipv4.conf.lo.promote_secondaries = 1
net.ipv4.ping_group_range = 0 2147483647
net.core.default_qdisc = fq_codel
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
fs.protected_regular = 2
fs.protected_fifos = 1
vm.max_map_count = 1048576
kernel.pid_max = 4194304
net.ipv4.ip_forward = 1
```

Pour permettre aux clients VPN d'accéder au réseau interne de l'entreprise, j'ai mis en place une règle de NAT (Network Address Translation) avec la commande `iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o enp0s3 -j MASQUERADE`. Cette règle masque les adresses IP des clients VPN derrière l'interface réseau du serveur, ce qui permet le routage vers le réseau interne. J'ai ensuite vérifié la configuration et son fonctionnement avec la commande `iptables -t nat -L -n -v`.

Mission 3 : Dispositifs de sécurité SSH et Fail2ban

Contexte :

1) Classement des outils selon leur rôle

- IDS (Intrusion Detection System) : outil qui détecte les tentatives d'intrusion ou comportements suspects sur le réseau.

- IPS (Intrusion Prevention System) : outil qui bloque automatiquement les attaques détectées.

- Outil de connexion distante : par exemple SSH ou VPN, permettant de se connecter à distance à un serveur ou réseau de façon sécurisée.

2) Différences entre IDS et IPS

Un IDS se contente de détecter et signaler les intrusions, mais n'intervient pas directement pour les bloquer.
Un IPS, lui, bloque activement les attaques en cours en plus de les détecter.
L'IPS est donc une version « active » de l'IDS.

3) Différences entre SSH et VPN

SSH permet d'établir une connexion sécurisée à distance vers un serveur spécifique, généralement pour administrer ou transférer des fichiers.
VPN crée un tunnel sécurisé entre deux réseaux ou entre un client et un réseau, donnant accès à l'ensemble du réseau comme si l'on était connecté localement.

4) Source de données utilisée par Fail2ban

Fail2ban fonctionne à partir des logs systèmes et logs des services (comme SSH, FTP, Apache…), qu'il analyse pour détecter des tentatives de connexion répétées ou suspectes.

5) Application utilisée par Fail2ban pour « blacklister » les IP

Fail2ban utilise iptables pour bloquer (blacklister) les adresses IP jugées malveillantes, en ajoutant des règles dans le firewall du serveur.

J'ai installé SSH ainsi que les services associés sur le serveur. Cette installation me permet de me connecter à distance de façon sécurisée pour administrer le serveur.

```
root@Srv-linux:~# ls /etc/ssh
moduli      ssh_config.d  sshd_config.d      ssh_host_ecdsa_key.pub  ssh_host_ed25519_key.pub  ssh_host_rsa_key.pub
ssh_config  sshd_config   ssh_host_ecdsa_key  ssh_host_ed25519_key    ssh_host_rsa_key
root@Srv-linux:~#
```

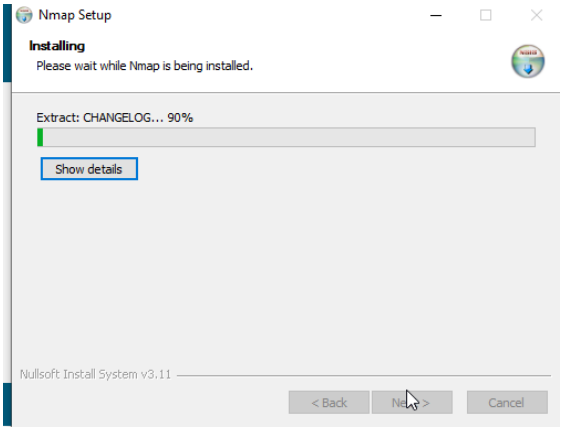
J'ai modifié le **port SSH** dans le fichier `/etc/ssh/ssh_config` afin de renforcer la sécurité des connexions à distance en utilisant un port personnalisé.

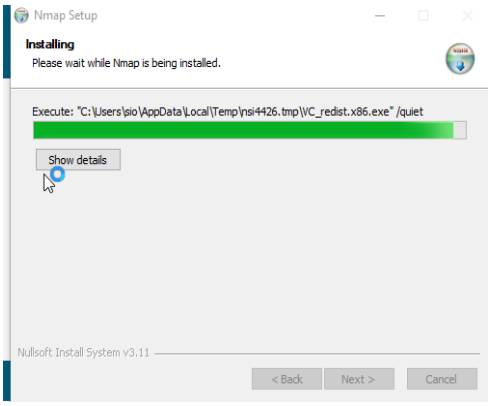
```
Include /etc/ssh/sshd_config.d/*.conf

Port 2222
```

J'ai effectué un **test de connexion SSH** pour vérifier que l'accès à distance fonctionne correctement avec la nouvelle configuration et le port modifié.

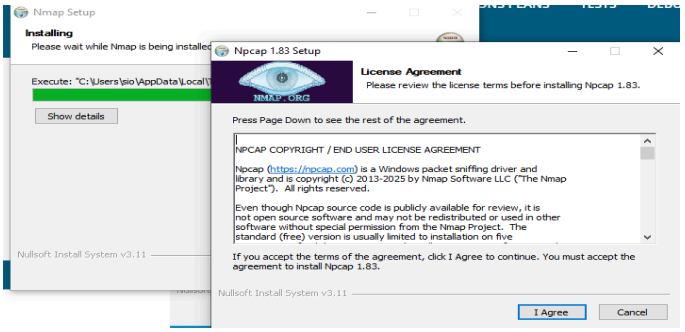
```
root@Srv-linux:~# ss -tlnp | grep ssh
LISTEN 0      128          0.0.0.0:2222      0.0.0.0:*    users:(("sshd",pid=833,fd=6))
LISTEN 0      128             [::]:2222         [::]:*    users:(("sshd",pid=833,fd=7))
root@Srv-linux:~#
```

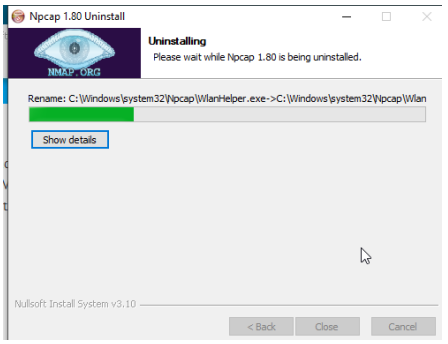
J'ai installé **Nmap** sur la machine cliente **Windows 10** afin de pouvoir analyser le réseau et vérifier la disponibilité des ports et services du serveur.





J'ai installé la **librairie Npcap**, nécessaire pour que Nmap puisse capturer et analyser les paquets réseau sur la machine cliente.





J'ai ouvert **PowerShell en mode administrateur** sur la machine cliente afin de pouvoir exécuter les commandes réseau avec les droits nécessaires.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> nmap -p 2222  192.168.13.3
```

J'ai observé que la commande `nmap -p 2222 192.168.13.3` n'était pas reconnue dans PowerShell, car je n'étais pas dans le répertoire contenant Nmap. Après m'y être placé, la commande a parfaitement fonctionné. J'ai également constaté qu'il fallait toujours ajouter `.\` devant `nmap` pour l'exécuter correctement dans ce répertoire.

```
PS C:\Windows\system32> cd "C:\Program Files (x86)\Nmap"
PS C:\Program Files (x86)\Nmap> .\nmap -p 2222 192.168.13.3
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-25 16:22 +0100
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-ser
vers
Nmap scan report for 192.168.13.3
Host is up (0.0011s latency).

PORT     STATE SERVICE
2222/tcp open  EtherNetIP-1

Nmap done: 1 IP address (1 host up) scanned in 0.80 seconds
PS C:\Program Files (x86)\Nmap>
```

La commande qui permet de vérifier la même chose directement sur le serveur Linux est :
`ss -tlnp | grep ssh`

J'ai réalisé un scan de port avec Nmap en utilisant la commande .\nmap -p 2222 192.168.13.3 afin de vérifier l'ouverture du port SSH modifié.

```
PS C:\Program Files (x86)\Nmap> .\nmap 192.168.13.3
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-25 16:35 +0100
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-ser
vers
Nmap scan report for 192.168.13.3
Host is up (0.0013s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE
111/tcp  open  rpcbind
2222/tcp open  EtherNetIP-1
3306/tcp open  mysql
5666/tcp open  nrpe

Nmap done: 1 IP address (1 host up) scanned in 0.30 seconds
PS C:\Program Files (x86)\Nmap>
```

Avec Nmap, j'ai pu observer les **services accessibles**, par exemple le port SSH `2222/tcp` ou le port MySQL `3306/tcp`. Les premières lignes indiquent la version de Nmap, et il est également possible de vérifier si le **DNS** est activé ou désactivé, ainsi que l'état de la machine, si elle est **UP** ou **DOWN**.

De retour sur le serveur, j'ai procédé à la **création des clés publiques et privées**, nécessaires pour sécuriser l'accès SSH.

```
root@Srv-linux:~# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519): _
```

J'ai intégré une **passphrase** dans le fichier de stockage pour SSH en utilisant `"je suis en sio"` pour l'utilisateur `ssh-sio`. Cela permet de sécuriser davantage la clé privée avec un mot de passe.

```
root@Srv-linux:~# ssh-keygen -b 4096
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519): ssh-sio
Enter passphrase for "ssh-sio" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ssh-sio
Your public key has been saved in ssh-sio.pub
The key fingerprint is:
SHA256:CQlnNELPCzXU7D7Kt7+cGkEjcwfKYusyvDhTlDBwgmQ root@Srv-linux
The key's randomart image is:
+--[ED25519 256]--+
|*E..+oOo.        |
|=o   X =o.       |
| o .+ X.+ .      |
|  o. + B.+       |
| .  . ..S        |
| ...    o.       |
| .+ .. ...       |
|o. +  o .o .     |
|.o.    .o+=.     |
+----[SHA256]-----+
root@Srv-linux:~#
```

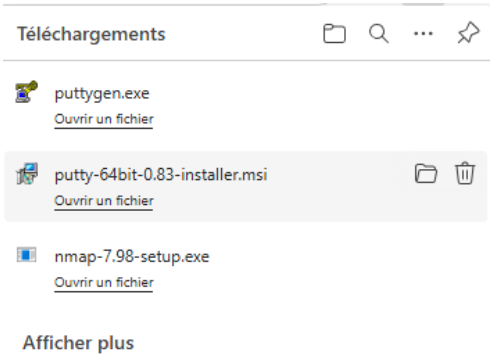
J'ai pu vérifier que mes **clés SSH** étaient présentes dans le répertoire `~/.ssh`, où elles sont stockées pour sécuriser les connexions à distance.

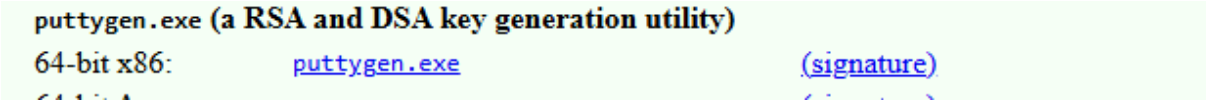
```
root@Srv-linux:~# ls -al .ssh
total 8
drwx------ 2 root root 4096  3 sept.  2024 .
drwx------ 4 root root 4096 25 nov.  17:04 ..
```

J'ai effectué un **test de connexion SSH** depuis mon poste Ubuntu vers mon serveur Linux à l'adresse `192.168.13.3` afin de vérifier que l'accès à distance fonctionne correctement avec les clés et la configuration mises en place.

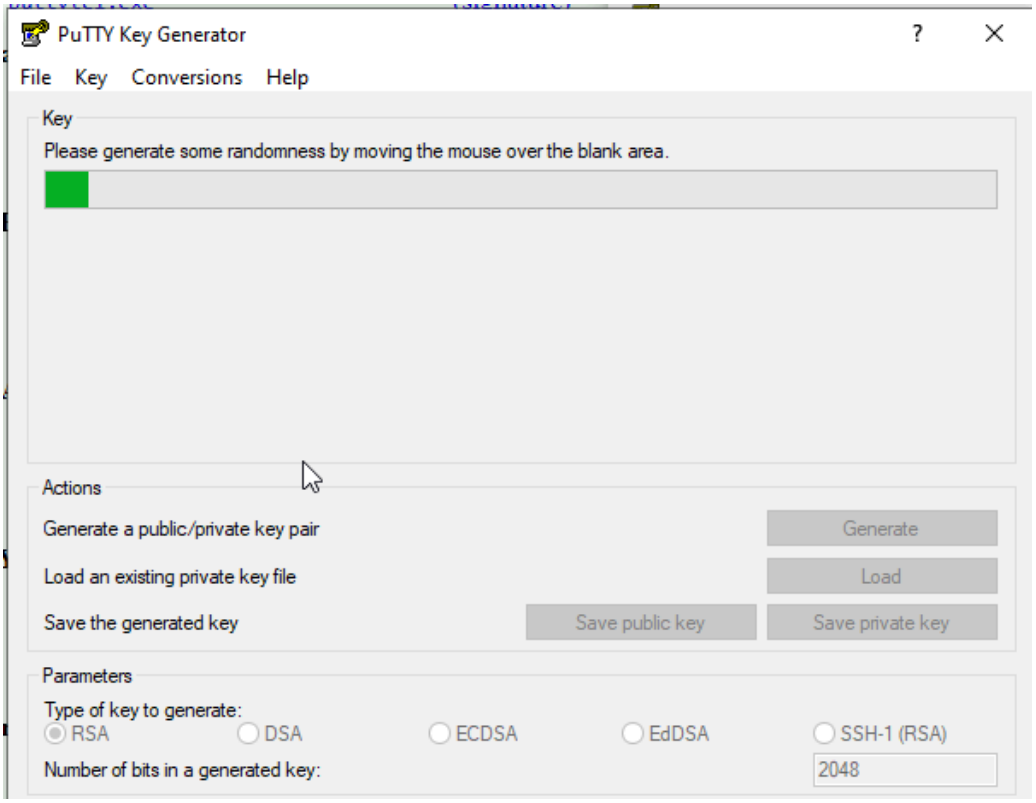
```
sio@C419-52:~$ sudo -i
[sudo] Mot de passe de sio :
root@C419-52:~# ssh sio@192.168.13.3 -p 2222
The authenticity of host '[192.168.13.3]:2222 ([192.168.13.3]:2222)' can't be es
tablished.
ED25519 key fingerprint is SHA256:PpuM17hi1M6thT218VvcSwRGqf8v3JzLOjcJSEFl+XI.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:7: [hashed name]
    ~/.ssh/known_hosts:10: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.13.3]:2222' (ED25519) to the list of known
hosts.
sio@192.168.13.3's password:
Linux Srv-linux 6.12.41+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.41-1 (202
5-08-12) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov  7 14:10:20 2025 from 192.168.13.250
sio@Srv-linux:~$
```

J'ai installé **PuTTYgen** sur la machine cliente Windows afin de générer et gérer les clés SSH pour établir des connexions sécurisées vers le serveur.



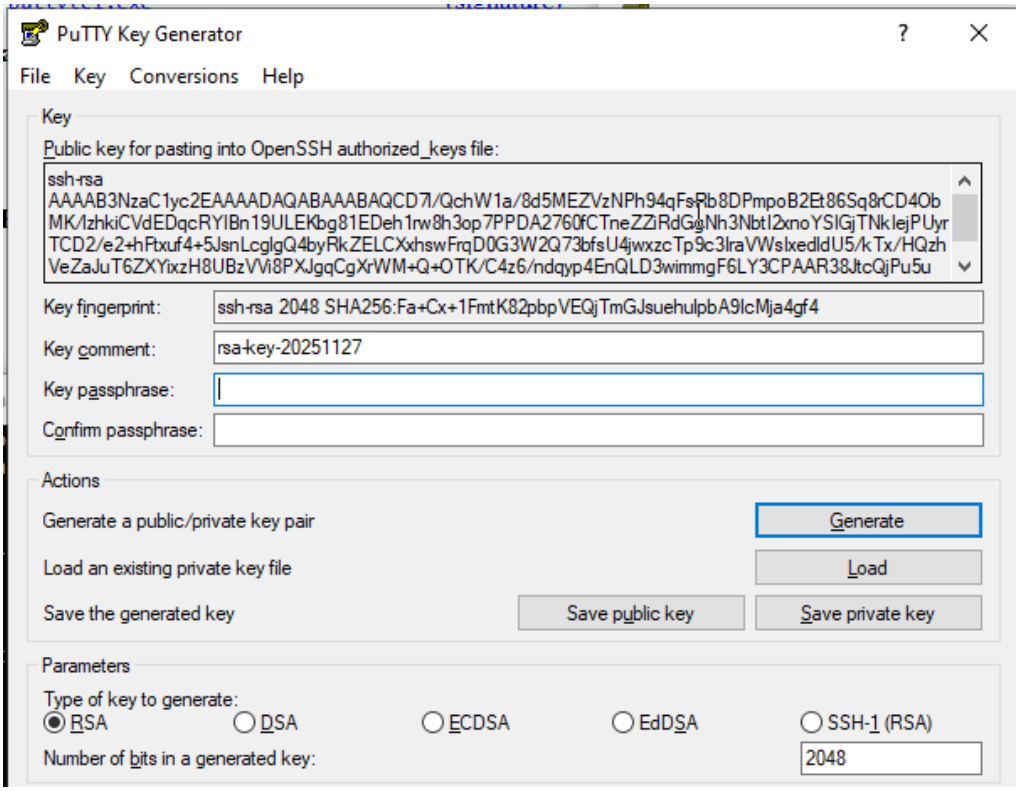


J'ai généré une **clé SSH** dans PuTTYgen en choisissant le **type RSA**. Cette clé servira à sécuriser l'authentification lors des connexions à distance vers le serveur.

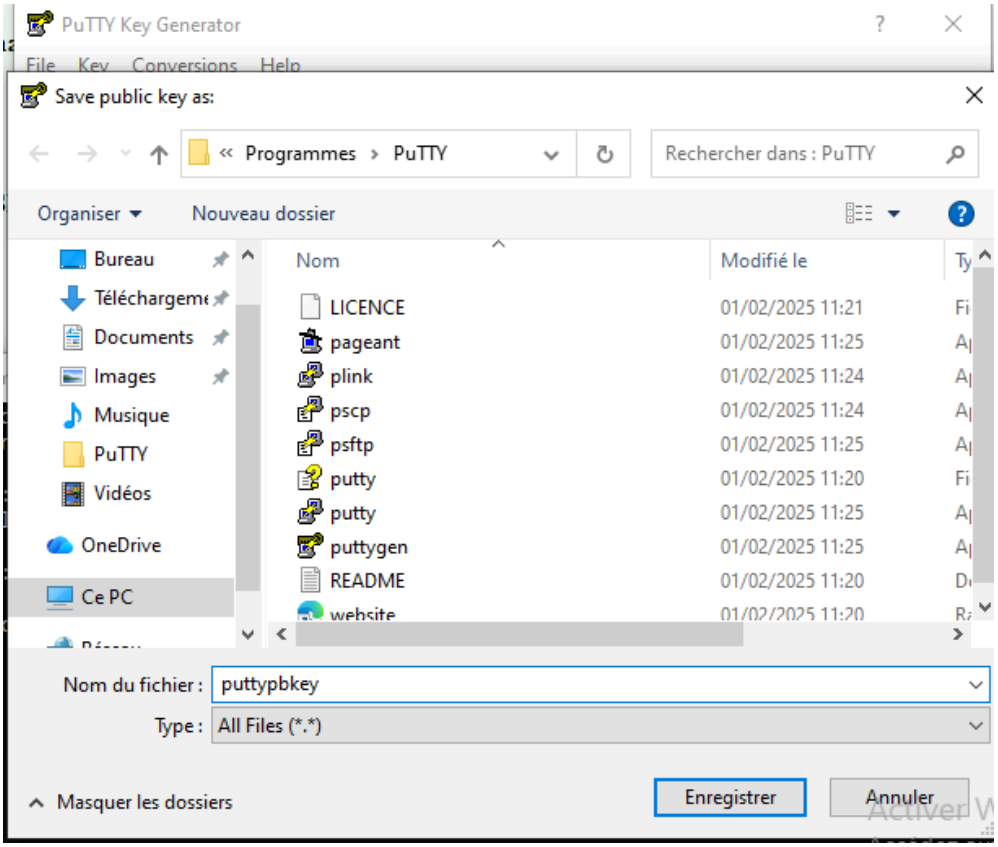


Juste en dessous, j'ai pu voir la clé SSH générée par PuTTYgen, prête à être utilisée pour l'authentification sécurisée sur le serveur.

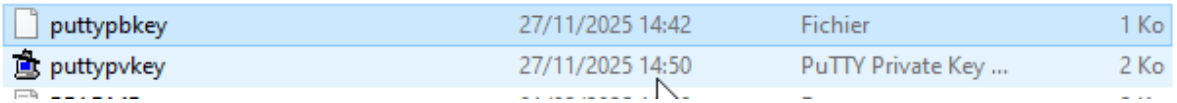
J'ai ajouté une passphrase à la clé SSH générée dans PuTTYgen en utilisant `"je suis en sio"` pour sécuriser davantage la clé privée.



J'ai enregistré la **clé publique** générée dans PuTTYgen sous le nom `puttypbkey`, afin de pouvoir l'utiliser pour configurer l'authentification SSH sur le serveur.



J'ai enregistré la **clé privée** générée dans PuTTYgen sous le nom `puttypvkey`, en veillant à la protéger avec la passphrase pour sécuriser l'accès SSH au serveur.



Fail2ban:

J'ai installé les paquets Fail2ban sur le serveur pour détecter et bloquer automatiquement les tentatives de connexion suspectes, renforçant ainsi la sécurité des services exposés.

```
root@postfix:~# apt-get install fail2ban
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libcbor0.8 libcurl3t64-gnutls libfuse2 libicu72 libldap-2.5-0 libnghttp3-9 libngtcp2-16 libngtcp2-crypto-gnutls8 libpe
```

J'ai modifié le fichier de configuration de Fail2ban pour protéger les connexions SSH. J'y ai défini le port SSH, le filtre, le chemin des logs pour conserver les traces, le nombre maximum de tentatives autorisées et la durée de bannissement d'une adresse IP en cas de comportement suspect.

```
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 86400
~
```

J'ai surveillé le fonctionnement de Fail2ban pour m'assurer qu'il détecte correctement les tentatives de connexion suspectes et qu'il bloque les adresses IP conformément à la configuration définie.

```
"/etc/fail2ban/jail.d/sshd.conf" [Nouveau] 7L, 104B écrit(s)
root@postfix:~# fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
root@postfix:~# journalctl  -u fail2ban
nov. 28 15:09:22 postfix systemd[1]: Started fail2ban.service - Fail2Ban Service.
nov. 28 15:09:23 postfix fail2ban-server[15941]: Server ready
root@postfix:~# tail -f /var/log/fail2ban.log
2025-11-28 15:09:23,015 fail2ban.jail           [15941]: INFO    Jail 'sshd' uses systemd {}
2025-11-28 15:09:23,020 fail2ban.jail           [15941]: INFO    Initiated 'systemd' backend
2025-11-28 15:09:23,021 fail2ban.filter         [15941]: INFO      maxLines: 1
2025-11-28 15:09:23,053 fail2ban.filtersystemd  [15941]: INFO    [sshd] Added journal match for: '_SYSTEMD_UNIT=ssh.serv
2025-11-28 15:09:23,053 fail2ban.filter         [15941]: INFO      maxRetry: 5
2025-11-28 15:09:23,054 fail2ban.filter         [15941]: INFO      findtime: 600
2025-11-28 15:09:23,054 fail2ban.actions        [15941]: INFO      banTime: 600
2025-11-28 15:09:23,054 fail2ban.filter         [15941]: INFO      encoding: UTF-8
2025-11-28 15:09:23,056 fail2ban.jail           [15941]: INFO    Jail 'sshd' started
2025-11-28 15:09:23,288 fail2ban.filtersystemd  [15941]: INFO    [sshd] Jail is in operation now (process new journal en
```

J'ai utilisé fail2ban-client pour consulter l'état du service. Il m'a permis de voir les jails actifs, c'est-à-dire les règles en place pour protéger différents services et les adresses IP actuellement surveillées ou bannies.

```
etc/fail2ban/jail.d/sshd.conf" 10L, 198B écrit(s)
root@postfix:~# tail -f /var/log/fail2ban.log
2025-11-28 15:09:23,015 fail2ban.jail           [15941]: INFO    Jail 'sshd' uses systemd {}
2025-11-28 15:09:23,020 fail2ban.jail           [15941]: INFO    Initiated 'systemd' backend
2025-11-28 15:09:23,021 fail2ban.filter         [15941]: INFO      maxLines: 1
2025-11-28 15:09:23,053 fail2ban.filtersystemd  [15941]: INFO    [sshd] Added journal match for: '_SYSTEMD_
2025-11-28 15:09:23,053 fail2ban.filter         [15941]: INFO      maxRetry: 5
2025-11-28 15:09:23,054 fail2ban.filter         [15941]: INFO      findtime: 600
2025-11-28 15:09:23,054 fail2ban.actions        [15941]: INFO      banTime: 600
2025-11-28 15:09:23,054 fail2ban.filter         [15941]: INFO      encoding: UTF-8
2025-11-28 15:09:23,056 fail2ban.jail           [15941]: INFO    Jail 'sshd' started
2025-11-28 15:09:23,288 fail2ban.filtersystemd  [15941]: INFO    [sshd] Jail is in operation now (process r
^C
root@postfix:~# fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:   sshd
root@postfix:~# fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
root@postfix:~#
```

J'ai configuré Fail2ban pour protéger les services du serveur. Cela inclut la définition des jails, des filtres, du nombre maximum de tentatives de connexion, du port SSH, du chemin des logs et de la durée de bannissement des adresses IP malveillantes.

```
 Notes.: Set the socket file. This is used to communicate with the daemon. Do
         not remove this file when Fail2ban runs. It will not be possible to
         communicate with the server afterwards.
 Values: [ FILE ]  Default: /var/run/fail2ban/fail2ban.sock

ocket = /var/run/fail2ban/fail2ban.sock

 Option: pidfile
 Notes.: Set the PID file. This is used to store the process ID of the
         fail2ban server.
 Values: [ FILE ]  Default: /var/run/fail2ban/fail2ban.pid

idfile = /var/run/fail2ban/fail2ban.pid

 Option: allowipv6
 Notes.: Allows IPv6 interface:
         Default: auto
 Values: [ auto yes (on, true, 1) no (off, false, 0) ] Default: auto
allowipv6 = auto

 Options: dbfile
 Notes.: Set the file for the fail2ban persistent data to be stored.
         A value of ":memory:" means database is only stored in memory
         and data is lost when fail2ban is stopped.
         A value of "None" disables the database.
 Values: [ None :memory: FILE ] Default: /var/lib/fail2ban/fail2ban.sqlite3
bfile = /var/lib/fail2ban/fail2ban.sqlite3

 Options: dbpurgeage
 Notes.: Sets age at which bans should be purged from the database
 Values: [ SECONDS ] Default: 86400 (24hours)
bpurgeage = 1d

 Options: dbmaxmatches
 Notes.: Number of matches stored in database per ticket (resolvable via
         tags <ipmatches>/<ipjailmatches> in actions)
 Values: [ INT ] Default: 10
bmaxmatches = 10

Definition]


Thread]

 Options: stacksize
 Notes.: Specifies the stack size (in KiB) to be used for subsequently created threa
         and must be 0 or a positive integer value of at least 32.
 Values: [ SIZE ] Default: 0 (use platform or configured default)
stacksize = 0
/etc/fail2ban/fail2ban.conf" 92L, 3015B
```

J’ai surveillé les logs avec Fail2ban pour vérifier que les tentatives de connexion suspectes étaient correctement détectées et enregistrées, et que les adresses IP malveillantes étaient bannies selon la configuration.

```
root@postfix:~# tail -f /var/log/fail2ban.log
2025-11-28 15:09:23,056 fail2ban.jail           [15941]: INFO    Jail 'sshd' started
2025-11-28 15:09:23,288 fail2ban.filtersystemd  [15941]: INFO    [sshd] Jail is in operation now (process new journal entries)
2025-11-28 16:11:51,514 fail2ban.server         [15941]: INFO    Shutdown in progress...
2025-11-28 16:11:51,514 fail2ban.observer       [15941]: INFO    Observer stop ... try to end queue 5 seconds
2025-11-28 16:11:51,535 fail2ban.observer       [15941]: INFO    Observer stopped, 0 events remaining.
2025-11-28 16:11:51,575 fail2ban.server         [15941]: INFO    Stopping all jails
2025-11-28 16:11:51,840 fail2ban.actions        [15941]: NOTICE  [sshd] Flush ticket(s) with nftables
2025-11-28 16:11:51,841 fail2ban.jail           [15941]: INFO    Jail 'sshd' stopped
2025-11-28 16:11:51,876 fail2ban.database       [15941]: INFO    Connection to database closed.
2025-11-28 16:11:51,876 fail2ban.server         [15941]: INFO    Exiting Fail2ban
```

J’ai installé iptables sur le serveur et vérifié son état pour m’assurer que le pare-feu est actif et prêt à gérer les règles de filtrage et de NAT pour sécuriser le réseau.

```
   32  2496 DROP       all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-skip-to-policy-output (0 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-track-forward (1 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-track-input (1 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-track-output (1 references)
 pkts bytes target     prot opt in     out     source               destination
   28  1680 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
 6704  713K ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-user-forward (1 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-user-input (1 references)
 pkts bytes target     prot opt in     out     source               destination
   73  4300 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0
   38  2280 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     udp  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-user-limit (0 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 LOG        all  --  *      *       0.0.0.0/0            0.0.0.0/0

    0     0 REJECT     all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-user-limit-accept (0 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain ufw-user-logging-forward (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-user-logging-input (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-user-logging-output (0 references)
 pkts bytes target     prot opt in     out     source               destination

Chain ufw-user-output (1 references)
 pkts bytes target     prot opt in     out     source               destination
root@postfix:~#
```